# SapphMap: A Differentially Oblivious Map for Enclaves

XIAOYU HAN, The Hong Kong University of Science and Technology, China
KE YI*, The Hong Kong University of Science and Technology, China
WEI DONG, Nanyang Technological University, Singapore
XINYING YANG, ByteDance, USA

Hardware enclaves securely process encrypted cloud data but remain vulnerable to access-pattern side-channel attacks. To address this, the access patterns must be made oblivious. While Full Obliviousness (FO) enforces input-independence, its high overhead has motivated Differential Obliviousness (DO), which relaxes this notion to a Differential Privacy guarantee. In this context, enclave-based oblivious maps maintain outsourced data in key-value stores while protecting accesses to internal enclave memory and external untrusted storage. However, existing FO designs are costly, while DO solutions largely target RAM-bound or static settings, making them insufficient for dynamic, large-scale workloads in practice.

In this paper, we present SapphMap, the first dynamic, external-memory DO map for enclaves that achieves high efficiency and supports range queries. SapphMap integrates an LSM-inspired hierarchy with specialized DO building blocks, adapting the Private Partition mechanism to construct partition-based private data structures for obfuscating access patterns. We prove it theoretically matches the write and query I/O complexity of standard non-oblivious leveled LSM-trees and empirically outperforms its FO counterparts, ENIGMAP and H2O2RAM, by 96× and 18.2× in time on real workloads, respectively, with only 0.57% and 38% of their storage.

CCS Concepts: • **Security and privacy → Management and querying of encrypted data**.

Additional Key Words and Phrases: differential obliviousness, hardware enclaves, oblivious maps

**ACM Reference Format:**
Xiaoyu Han, Ke Yi, Wei Dong, and Xinying Yang. 2026. SapphMap: A Differentially Oblivious Map for Enclaves. *Proc. ACM Manag. Data* 4, 6 (SIGMOD), Article 375 (December 2026), 36 pages. https://doi.org/10.1145/3856370

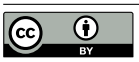
## 1 Introduction

Outsourcing data to the cloud has become a prevalent practice in industry [43]. Since such data often contains sensitive information, it is typically encrypted prior to storage. In a traditional client-server setting, however, operating on encrypted data requires frequent network round-trips to retrieve, process, and re-encrypt the data at the trusted client [73], incurring substantial communication overhead and client-side burden. To bypass these bottlenecks, hardware enclaves (e.g., Intel SGX [17, 28], RISC-V Keystone [55]) have increasingly been deployed in the cloud as trusted proxies, protecting sensitive code and data within a hardware-isolated memory region.

Despite external data encryption and isolated execution, enclaves remain vulnerable to side-channel attacks [15, 53, 78], as their access patterns (i.e., the ordered sequence of addresses in enclave memory and external storage) are still visible to the untrusted host. This leakage can be exploited

*Corresponding author.

Authors' Contact Information: Xiaoyu Han, The Hong Kong University of Science and Technology, Hong Kong, China, xhanbh@connect.ust.hk; Ke Yi, The Hong Kong University of Science and Technology, Hong Kong, China, yike@cse.ust.hk; Wei Dong, Nanyang Technological University, Singapore, Singapore, wei_dong@ntu.edu.sg; Xinying Yang, ByteDance, San Jose, USA, xinying.yang@bytedance.com.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2836-6573/2026/12-ART375
https://doi.org/10.1145/3856370

by an adversary to infer sensitive information [49, 83] and, in severe cases, even reconstruct the entire database [39, 52]. To mitigate this threat, a standard approach is to render access patterns oblivious. The strongest notion, *full obliviousness (FO)* [34], enforces data-independent access patterns and provides strict privacy protection, yet this guarantee often incurs prohibitive overhead in practice. As a more efficient alternative, *differential obliviousness (DO)* [9] requires access patterns on neighboring inputs to satisfy differential privacy (DP), thus protecting the privacy of individuals in the database while improving efficiency.

Building on these paradigms, enclave-based *oblivious maps* [29, 62, 75, 80] have been proposed as a core abstraction to maintain encrypted outsourced data in a key-value (KV) store, a storage model implemented by widely used engines such as Pebble [60] and RocksDB [22], which underpin enterprise-grade DBMSs like CockroachDB [74] and TiDB [48], respectively. These maps must protect access patterns while memory state persists across operations. Moreover, this protection covers two levels: access patterns within enclave memory and across external untrusted storage, a notion called *strong obliviousness* [10, 68] or *double obliviousness* [62, 75].

**Design Challenges.** However, it is non-trivial to devise a practical doubly-oblivious map, as it must jointly satisfy several desiderata:

*(1) External I/O Efficiency.* Real-world outsourced datasets are often too large to fit entirely in enclave memory. Although modern platforms such as Intel SGX v2 [28] support much larger enclaves (e.g., 512GB per socket), practical deployments are still constrained by the host's physical memory capacity. Consequently, processing such data inevitably incurs frequent accesses to untrusted memory and may even spill to disk. This calls for an I/O-efficient design that remains effective under realistic hardware constraints.

*(2) Dynamic Data Maintenance.* In practice, KV stores are rarely static; instead, they evolve through a continuous stream of insertions, updates, and deletions. Hence, the underlying structure must handle dynamic operations over persistent state without frequent global rebuilding, whose overhead is prohibitive.

*(3) Range Query Support.* In addition to simple point lookups, many cloud-based applications also require retrieving records over specific key intervals, such as temporal logs [8] or ledger entries [89]. Explicit support for range queries significantly broadens the system's functionality beyond basic oblivious lookup operations.

Taken together, these requirements pose formidable technical challenges for designing a practical doubly-oblivious map. Specifically, maintaining large-scale, dynamic data on untrusted storage while enforcing oblivious access patterns creates an inherent tension with performance. Furthermore, efficient range queries typically rely on order-aware physical layouts to maximize spatial locality, yet this is difficult to balance with oblivious writes, which must conceal correlations between data and its physical placement.

**Limitations of Prior Solutions.** Existing doubly-oblivious designs attempt to navigate these competing demands, but all have to make unfavorable compromises. ENIGMAP [75] extends an oblivious AVL tree [80] with a locality-friendly ORAM layout to realize a dynamic external-memory FO map. More recently, H2O2RAM [86] combines hierarchical ORAM with tailored oblivious hash schemes for better data locality and parallelization, which readily provides FO map support. However, both schemes incur substantial overhead because of their FO guarantees, and their layouts do not naturally support efficient range queries. In the DO context, Chan et al. [9] combine noisy-boundary binning with DP interior points [6] to construct the first DO range-query data structure. Nevertheless, this design is insertion-only and based on the RAM model; if adapted for external I/O, it would require $\Omega(\log^9 N)$ enclave memory to function and a block size of the same asymptotic order for favorable query I/O, making it a result of theoretical interest only. Doquet [66]

Table 1. **Comparison of SapphMap and representative schemes.** We compare privacy guarantees (Priv.), range-query support (RQ), external-memory (EM) model consideration, (amortized) write and query complexities, and system requirements. NO-LSMTree is a non-oblivious leveled LSM-tree baseline for enclaves. See Table 2 for notation and Section 3.4.1 for Sort($N$).

| Scheme | Priv. | RQ | EM | Per-Write I/O | Per-Write Comp. | Query I/O① | System Requirements |
|---|---|---|---|---|---|---|---|
| NO-LSMTree | None | ✓ | ✓ | $O(\frac{\gamma}{B}\log_\gamma \frac{N}{M})$ | $O(\gamma \log_\gamma N)$ | $O(\log_\gamma \frac{N}{M} + \frac{R}{B})$ | $M = \Omega(B)$ |
| SapphMap (Ours) | DO | ✓ | ✓ | $O(\frac{\gamma}{B}\log_\gamma \frac{N}{M})$ | $O(\gamma \log_\gamma N \log\log D)$ | $O(\log_\gamma \frac{N}{M} + \frac{R}{B})$ | $M = \Omega(B\log N), B = \Omega(\log^2 D)$③ |
| ENIGMAP [75] | FO | × | ✓ | $O(\log_B N \log N)$ | $\tilde{O}(\log^2 N)$ | $O(\log_B N \log N)$② | $M = \Omega(B\log B)$ |
| H2O2RAM [86] | FO | × | × | $O(\frac{\text{Sort}(N)}{N}\log N)$④ | $\tilde{O}(\log^2 N)$ | $O(\frac{\text{Sort}(N)}{N}\log N)$②④ | $M = \Omega(B^2), B = \Omega(\log^2 N)$④ |
| Chan et al. [9] | DO | ✓ | × | $O(\frac{1}{B}\log N)$④ | $O(\log N \log\log N)$ | $O(\log N + \frac{R}{B})$④ | $M = \Omega(B), B = \Omega(\log^9 N)$④ |
| Doquet [66] | DO | ✓ | × | N/A (Static) | N/A (Static) | $O(\log_B \frac{N}{M} + \frac{R}{B})$④ | $M = \Omega(B), B = \Omega(\log N)$④ |

① We omit the computation cost for queries in this table; detailed results for SapphMap are provided in Sections 6.2 and 6.3.
② ENIGMAP and H2O2RAM lack range-query support, so their query I/O is for point queries only. Other schemes support both, treating point queries as a special case with $R = 1$.
③ This holds for constant $\gamma$ under dynamic writes, as is standard practice for LSM-trees and our SapphMap (with $\gamma$ typically 2 to 16). We analyze the general case for $\gamma$ in Section 6.4.
④ These I/O complexities were not explicitly stated in the original works due to their TEE-based RAM settings, but follow directly from their analyses when applied to EM.

further proposes DO private data structures based on DP histograms [45] for efficient oblivious range and join queries, but its design is limited to static datasets. In addition, it does not discuss external I/O efficiency, and its evaluation assumes the entire dataset fits within enclave memory. Table 1 compares these schemes.

**Our Work.** In this paper, we propose SapphMap, the first practical doubly-DO map for enclaves that fulfills all three critical desiderata. Formally, as in prior work [9], it guarantees that the executions (capturing both access patterns and function leakage) induced by neighboring write sequences (i.e., insertions, updates, and deletions) satisfy DP. At its core, SapphMap integrates a lazy, LSM-inspired hierarchical architecture with carefully designed DO building blocks. The key insight is to leverage the Private Partition mechanism [25, 67] to partition the dataset into segments, which are subsequently padded to a uniform upper-bound size. This process transforms fine-grained item-level traces into coarse-grained segment-level accesses. As a result, the released DP segment boundaries obfuscate write access patterns and restrict query observability, thereby hiding whether any specific key exists.

We summarize our main contributions as follows:

- We design and implement SapphMap, the first dynamic, external-memory DO map for enclaves, which guarantees DP over both internal executions and external accesses.
- We adapt the Private Partition DP mechanism to construct a partition-based private data structure for DO data maintenance, integrating a suite of DO algorithms and query refinements like fractional cascading. With this design, SapphMap theoretically matches standard non-oblivious leveled LSM-trees in I/O complexity for amortized writes and queries, while incurring only a doubly logarithmic computational overhead per write.
- We provide a rigorous privacy budget allocation scheme for SapphMap and formally characterize its system parameter requirements, proving that the resulting block-size and enclave-memory constraints are practical for real-world deployments.
- Our extensive comparative experiments and parameter analyses show SapphMap scales well, supports efficient range queries, and balances costly FO against efficient but insecure designs. On two real workloads, it yields average speedups of 96× and 18.2× over FO baselines ENIGMAP and H2O2RAM, respectively, while using 0.57% and 38% of their space.

## 2 SapphMap Overview

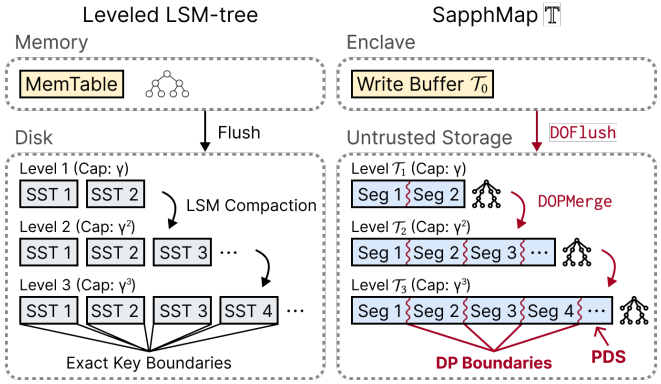



Fig. 1. LSM-inspired architecture of SapphMap.

Figure 1 illustrates the architecture of SapphMap and its connection to leveled LSM-trees. SapphMap adopts a Log-Structured Merge (LSM)-inspired hierarchical structure $\mathbb{T}=[\mathcal{T}_0,\ldots,\mathcal{T}_L]$. The in-enclave level $\mathcal{T}_0$ serves as a persistent write buffer, while each deeper level stores a partition-based Private Data Structure (PDS) that organizes items into padded segments via released DP boundaries. As in a leveled LSM-tree, the logical capacity grows across levels with fan-out $\gamma$, amortizing data reorganization across writes.

We initialize SapphMap via bulk loading, which constructs a PDS from the initial dataset and places it at the appropriate level of $\mathbb{T}$. For dynamic streams, our append-only model treats all KV writes as versioned entries accumulated in $\mathcal{T}_0$; once full, we build a PDS over its keys and flush it to $\mathcal{T}_1$ via `DOFlush`. For a deeper level $\mathcal{T}_i$, a direct migration or DO merge (`DOPMerge`) to $\mathcal{T}_{i+1}$ is triggered when $\mathcal{T}_i$ reaches its logical capacity $\gamma$, i.e., when $\gamma$ PDSs from the preceding level are merged. We also build per-level B+-trees over released DP boundaries to accelerate overlapping-segment lookups.

For point and range queries, we inspect the buffer and traverse the hierarchy via per-level B+-trees to locate overlapping segments, followed by an oblivious scan for results. Thus, buffered records are visible to subsequent queries without waiting for $\mathcal{T}_0$ to fill or flush. As a theoretical refinement, fractional cascading can reduce repeated per-level lookups in this procedure (Section 6.3).

This LSM-inspired organization alone, however, is insufficient for DO. Basic LSM-trees organize levels into SSTables by key ranges and data volumes; thus, flushes and compactions expose exact boundaries, true occupancies, and data-dependent I/O. This motivates dedicated DO algorithms to securely construct and merge partition-based PDSs during flushes and compactions (Section 5).

## 3 Preliminaries

To formalize the privacy requirements and support the subsequent DO algorithms, this section presents the core concepts and primitives. Table 2 summarizes the notations employed throughout the paper. Detailed proofs appear in the full version [44].

### 3.1 Differential Privacy

*Definition 3.1 (Differential Privacy [23, 26]).* A randomized mechanism $\mathcal{M}$ is $(\varepsilon,\delta)$-differentially private (DP) if for any neighboring datasets $\mathcal{D}_1,\mathcal{D}_2$, and any subset of outputs $S$:

$$\Pr[\mathcal{M}(\mathcal{D}_1)\in S]\le e^{\varepsilon}\cdot\Pr[\mathcal{M}(\mathcal{D}_2)\in S]+\delta,$$

where the probability Pr is over the internal randomness of $\mathcal{M}$, and $e$ is Euler's number. Typically, $\varepsilon=\Theta(1)$ and $\delta=O(N^{-c})\ll 1/N$ for a size-$N$ dataset and constant $c>1$. The case $\delta=0$ is called $\varepsilon$-DP.

Table 2. Summary of Important Notations

| | |
|---|---|
| **System and Workload** | |
| $M, B$ | Enclave memory size and transfer block size (in items), respectively |
| $D$ | Key space domain size (the domain is $[1, D]$) |
| $N$ | Dataset size or total number of write operations |
| $N_{\max}$ | Maximum number of writes, assuming $\log N_{\max} = \Theta(\log D)$ |
| $R$ | Number of items matching a range query |
| **DP Mechanisms and DO Algorithms** | |
| $\varepsilon_{\text{tot}}, \delta_{\text{tot}}$ | Total privacy budget for SapphMap |
| $\varepsilon, \delta$ | Privacy budget per DP mechanism or DO algorithm |
| $\zeta$ | Laplace noise bound w.h.p. and truncation range |
| $Z$ | Maximum segment size (used as the target padding size) |
| $\mathcal{B}$ | Transient in-enclave working buffer for DO algorithms |
| $\eta$ | Estimation error bound w.h.p. for DP prefix sums |
| $w$ | Batch size of segments processed in `DOItemAssign` |
| **SapphMap** | |
| $\mathcal{T}_i$ | $i$-th level of SapphMap ($\mathcal{T}_0$: persistent in-enclave buffer level) |
| $\mu$ | Maximum PDS constructions an item undergoes in SapphMap |
| $\gamma, \rho$ | Fan-out $\gamma$ of SapphMap, set as a constant or $B^\rho$ with $\rho \in (0, 1)$ |

In our context, each dataset is a set of KV pairs, and two datasets are neighbors if they differ in one key. The associated values are irrelevant to DP, since access pattern does not depend on them.

*Definition 3.2 ($\ell_1$-sensitivity [23, 24]).* The $\ell_1$-sensitivity of a query $f : \mathcal{D} \to \mathbb{R}^d$ is defined as:

$$\Delta_1 f = \max_{\mathcal{D}_1, \mathcal{D}_2} \|f(\mathcal{D}_1) - f(\mathcal{D}_2)\|_1,$$

where the max is taken over all neighboring datasets $\mathcal{D}_1, \mathcal{D}_2$.

Proposition 3.3 (Post-Processing [26]). *Suppose $\mathcal{M}$ is an $(\varepsilon, \delta)$-DP mechanism and $g$ is an arbitrary randomized mapping defined on $\mathcal{M}$'s output. Then, the composition $g \circ \mathcal{M}$ satisfies $(\varepsilon, \delta)$-DP.*

Theorem 3.4 (Composition [26, 27]). *Let $\{\mathcal{M}_i\}_{i=1}^m$ be a sequence of $(\varepsilon_i, \delta_i)$-DP mechanisms. Their sequential composition on the same dataset (where subsequent mechanisms may adaptively depend on prior outputs) satisfies $(\sum_{i=1}^m \varepsilon_i, \sum_{i=1}^m \delta_i)$-DP, whereas their parallel composition on disjoint subsets satisfies $(\max_{i=1}^m \varepsilon_i, \max_{i=1}^m \delta_i)$-DP.*

Theorem 3.5 (Group Privacy [26]). *If a mechanism $\mathcal{M}$ satisfies $(\varepsilon, \delta)$-DP, then for any datasets with distance $\leq k$ (i.e., differing by at most $k$ keys), $\mathcal{M}$ satisfies $(k\varepsilon, ke^{(k-1)\varepsilon}\delta)$-DP for groups of size $k$.*

### 3.2 Common DP Mechanisms

*Definition 3.6 (Laplace Mechanism [24]).* Let $f : \mathcal{D} \to \mathbb{R}^d$ be a query function with $\ell_1$-sensitivity $\Delta_1 f$. The mechanism $\mathcal{M}(\mathcal{D}) = f(\mathcal{D}) + (Y_1, \ldots, Y_d)$ preserves $\varepsilon$-DP, where $Y_1, \ldots, Y_d$ are i.i.d. random noise variables drawn from the Laplace distribution $\text{Lap}(\Delta_1 f/\varepsilon)$.

For our subsequent analysis, we further denote by $\text{Lap}_\zeta(b)$ the Laplace distribution with scale $b$ truncated to the interval $[-\zeta, \zeta]$.

Lemma 3.7 (Concentration Bound for Laplace Sums). *Let $Y_1, \ldots, Y_k$ be independent random variables drawn from $\text{Lap}(\lambda)$. For any failure probability $\beta \in (0, 1)$, the following tail bound holds:*

$$\Pr\left[\left|\textstyle\sum_{i=1}^k Y_i\right| > \alpha_{\text{Lap}}^{(k)}(\lambda, \beta)\right] \leq \beta,$$

$$\text{where } \alpha_{\text{Lap}}^{(k)}(\lambda, \beta) = \lambda \cdot \max\left\{2\sqrt{2k \ln(2/\beta)}, 4 \ln(2/\beta)\right\}.$$

The *Binary Mechanism* (BM) [11] extends the Laplace mechanism to release $\varepsilon$-DP prefix sums over a length-$T$ integer stream using a binary tree of noisy partial sums. Neighboring streams differ in one entry by at most 1. Any prefix query aggregates at most $\lceil \log_2 T \rceil$ nodes, while each input affects at most $\lfloor \log_2 T \rfloor + 1$ nodes. Thus:

Lemma 3.8 (Estimation Error of Binary Mechanism). *Let $\mathcal{F}$ (with entries $f_i$) be the true prefix sums of a length-$T$ data stream, and $\tilde{\mathcal{F}}$ be the noisy ones by $\varepsilon$-DP BM. Then w.p. at least $1-\beta$, the estimation error satisfies $|\tilde{f}_i - f_i| \le \alpha_{\mathrm{BM}}(T,\varepsilon,\beta)$ uniformly for all $i = 1,\ldots,T$, where*

$$\alpha_{\mathrm{BM}}(T,\varepsilon,\beta) = \alpha_{\mathrm{Lap}}^{(\lceil \log_2 T \rceil)}\left((\lfloor \log_2 T \rfloor + 1)/\varepsilon, \beta/T\right).$$

### 3.3 Differential Obliviousness

*Definition 3.9 (Differential Obliviousness [9, 90]).* A randomized mechanism $\mathcal{M}$ (resp. data structure $\mathbb{DS}$) satisfies $(\varepsilon,\delta)$-differential obliviousness (DO) if for any two neighboring datasets $\mathcal{D}_1, \mathcal{D}_2$ (resp. operation sequences $\vec{\sigma}_1, \vec{\sigma}_2$) and any subset of executions $S$:

$$\Pr\left[\mathsf{Exec}^{\mathcal{M}}(\mathcal{D}_1) \in S\right] \le e^{\varepsilon} \cdot \Pr\left[\mathsf{Exec}^{\mathcal{M}}(\mathcal{D}_2) \in S\right] + \delta,$$

where $\mathsf{Exec}^{\mathcal{M}}(\mathcal{D})$ (resp. $\mathsf{Exec}^{\mathbb{DS}}(\vec{\sigma})$) samples a random execution of $\mathcal{M}$ (resp. $\mathbb{DS}$) on input dataset $\mathcal{D}$ (resp. operation sequence $\vec{\sigma}$). Specifically, $\mathsf{Exec} \coloneqq (\mathsf{Access}, \mathsf{Leakage})$ captures the joint distribution of the access pattern and function leakage (which mainly includes the output size and the intentionally released data) w.r.t. the input. Neighboring inputs have the same size (resp. length) and differ in exactly one record (resp. operation).

By comparison, *full obliviousness* (FO) represents a stricter privacy requirement, demanding that the execution distributions be data-independent across all inputs of the same size (resp. length).

Prior DO definitions typically formulate privacy in terms of access patterns [9, 66], incorporating function leakage separately via composition lemmas. Our formulation is closer to [90], capturing access patterns together with function leakage in a unified execution footprint. In a compositional DO framework, upstream function leakage can directly shape both the immediate output trace and downstream access patterns. This yields a concise, self-contained formulation that naturally captures all observable information.

### 3.4 Algorithmic Primitives

This section introduces the algorithmic primitives that underpin oblivious internal operations in our DO algorithms. Certain primitives and the algorithms in Section 5 require specific block-size and enclave-memory constraints. For clarity, we take these constraints as given for now and defer the formal analysis to Section 6.4.

*3.4.1 Sorting.* We choose `Flexway-Osort` [41] as our FO sorting primitive[1]. For an input of $N$ items, it incurs $O(N \log N)$ computation and an I/O cost of $O(\mathrm{Sort}(N)) = O((N/B)\log_{M/B}(N/B))$, where $M$ is the enclave memory size and $B$ is the enclave transfer block size (both measured in items). We write $\mathtt{FOSort}(\mathcal{X}, \langle \mathcal{X}.k_i \circ_i \rangle)$ to denote sorting $\mathcal{X}$ by fields $\mathcal{X}.k_i$ with directions $\circ_i \in \{\uparrow, \downarrow\}$.

*3.4.2 Compaction.* The compaction primitive takes an array $\mathcal{X}$ of $N$ elements, each tagged with a binary flag. It permutes $\mathcal{X}$ such that all marked items (flag 1) precede the unmarked ones (flag 0). We employ `ORCOMPACT` [70] for FO compaction, which incurs $O(N \log N)$ computation and an I/O cost bounded by `FOSort` (as a sorting special case). For the DO variant, we utilize the Hamming-neighbor-based `TightStableCompact` [9], with $O(N \log\log N)$ computation and $O(N/B)$ I/O cost. The two algorithms, denoted $\mathtt{FOCompact}(\mathcal{X})$ and $\mathtt{DOCompact}(\mathcal{X})$, respectively, preserve marked-item order.

[1] We omit `Flexway-Osort`'s negligible failure probability from our privacy analysis.

*3.4.3 Distribution.* The distribution primitive takes an input array $\mathcal{X}$ and an injective mapping vector $\mathcal{V}$, both of size $N$. It constructs an output array of size $N_{\text{out}}$ ($N_{\text{out}} \geq N$) by placing each $\mathcal{X}[i]$ at the position specified by $\mathcal{V}[i]$. We use in-place `OBLIVIOUS-DISTRIBUTE` [54] for FO distribution, denoted $\texttt{FODistribute}(\mathcal{X}, \mathcal{V}, N_{\text{out}})$. It incurs $O(N_{\text{out}} \log N_{\text{out}})$ computation including input sorting.

## 4 Problem Definition

### 4.1 Data Outsourcing Model

We consider a secure data outsourcing model where a client offloads the storage and query processing of a sensitive dataset $\mathcal{D}$, modeled as a collection of KV pairs, to an untrusted server. Specifically, the client first bulk-loads an initial dataset to the server. It then issues a sequence of operations, each being either a write (inserting, updating, or deleting a KV pair) or a read (a point or range query). All client-server communication is encrypted to ensure confidentiality.

The server utilizes a hardware enclave, such as Intel SGX [17, 28] or RISC-V Keystone [55], to isolate code and data within a protected memory region (e.g., the Enclave Page Cache (EPC) in Intel SGX). It employs a secure swapping mechanism to transfer data blocks between the enclave and untrusted storage (memory or disk). During these transfers, data is decrypted solely upon enclave entry and re-encrypted with fresh randomness using an enclave-resident key prior to eviction. To facilitate efficient queries, the server maintains an encrypted data structure over $\mathcal{D}$ within untrusted storage. Figure 2 illustrates the overall secure data outsourcing model.

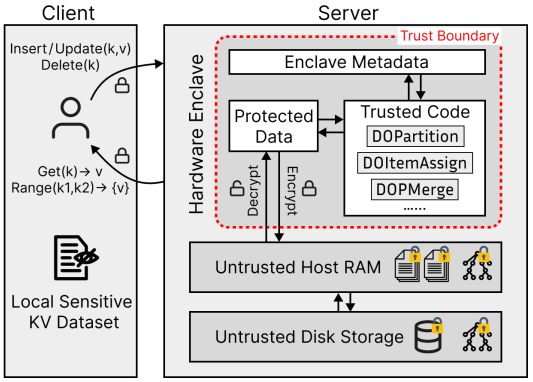



Fig. 2. Overview of the secure data outsourcing model

### 4.2 Threat Model

We model the Server as an honest-but-curious adversary controlling the software stack without physical enclave access. Unable to directly access or tamper with protected code and data, the adversary can observe external I/O across the enclave boundary, profile internal access patterns via side channels (e.g., cache attacks [5, 71, 78]), and obtain intentionally released function leakage to reconstruct execution footprints and infer secrets [56, 63, 83]. We exclude attacks like denial-of-service (DoS) [21, 51, 84] and timing attacks [19, 38], which are orthogonal to our focus on execution-footprint leakage and can be mitigated using complementary defenses [15, 31, 40].

Given this threat model, an ideal defense is a doubly-FO [62] data structure that supports efficient writes, plus point and range queries. To fully prevent secret information leakage, it must jointly protect both internal enclave execution and external data accesses. Due to its excessive overhead, however, we instead aim to design a doubly-DO alternative. This rigorously bounds information leakage on both fronts under a DP guarantee, ensuring individual-level privacy while achieving far better performance than its FO counterparts.

To this end, our DO definition is formulated over write sequences (i.e., two operation sequences are neighbors if they differ by a single write operation). This formulation protects the presence of individual records contributed by data owners, rather than the privacy of queries issued by data analysts, which aligns with established DP settings in applications such as census and legal databases [1, 32]. Accordingly, the adversary may learn query contents but cannot infer whether a specific key exists in $\mathcal{D}$. We further note that extending DO to queries would incur a steep $\Omega(N)$ cost per query [9], since a range query may return anywhere from 0 to all $N$ keys.

## 5 Private Data Structure Design

In this section, we detail DO algorithms for PDS construction and maintenance. We first review the original Private Partition mechanism in Section 5.1. Next, we introduce a DO partitioning algorithm in Section 5.2 to generate DP boundaries from input keys. We then present a DO method in Section 5.3 for assigning items to segments while concealing real item counts. Finally, we describe an efficient DO merge algorithm for PDSs in Section 5.4. For clarity, we use generic $(\varepsilon, \delta)$ notation for the privacy budget assigned to each primitive, while deferring both the budget allocation among subroutines and the composition across primitives to Section 6.1.3.

### 5.1 Review of Private Partition

The *Private Partition* mechanism [25, 67] partitions a 0–1 stream of length $T$ into contiguous segments with DP boundaries. Based on the Sparse Vector Technique (SVT) [26, 58], it identifies boundaries by comparing noisy cumulative counts with noisy thresholds.

*Example 5.1.* Consider the 0–1 stream $[0, 1, 0, 0, 1, 0, 0, 1, 0, 0]$ with threshold 2. At position 5 (prefix $[0, 1, 0, 0, 1]$), the count of 1-entries reaches 2. Suppose the noisy count (2.1) exceeds the noisy threshold (1.8); the mechanism outputs position 5 as a boundary and resets the count. If no further comparison succeeds on the suffix $[0, 0, 1, 0, 0]$, the resulting segments are $[1, 5]$ and $[6, 10]$.

Interpreting each 1-bit position as a key, the example stream encodes $\mathcal{X} = [2, 5, 8]$ over the domain $[1, 10]$. More generally, any set of $n$ sorted keys over $[1, D]$ can be represented as a length-$D$ 0–1 stream, enabling direct application of Private Partition. While a naive scan takes $O(D)$ time, Dwork et al. [25] process them in $\tilde{O}(n)$ time by geometrically sampling boundaries within empty intervals between adjacent keys, where the sealing probability is uniform.

In the resulting partition, non-final segment sizes (key counts per segment) show a concentration around $\Theta(\log D)$. More precisely,

Lemma 5.2 (Segment Size in Private Partition). *For an $\varepsilon$-DP Private Partition over $[1, D]$ with threshold $3\ln(2D/\beta)/\varepsilon$, w.p. at least $1 - \beta$, every non-final segment has a size between $\ln(2D/\beta)/\varepsilon$ and $Z = 5\ln(2D/\beta)/\varepsilon$. In particular, the expected segment size is $\Theta(Z)$.*

The resulting DP boundaries support our design by enabling later segment-level DO processing. However, the efficient implementation is not oblivious: observing repeated sampling in an adjacent-key gap reveals the presence of a boundary. The challenge is to eliminate this leakage while preserving partition guarantees.

### 5.2 DO Private Partition

We adapt Private Partition [25] to partition sorted, distinct keys $x_1 < x_2 < \ldots < x_n$ over $[1, D]$. In our global DO definition, neighboring inputs differ by one write operation, which changes the underlying key set by at most two keys and thus induces a Hamming distance of at most 2 between the corresponding 0–1 streams. By group privacy, calibrating Private Partition with $\varepsilon/2$ yields $\varepsilon$-DP.

To eliminate access-pattern leakage from the original mechanism, DOPartition draws exactly one sample per adjacent-key interval, rendering the sampling pattern data-independent. It further uses truncated Laplace noise for threshold checks to preserve utility.

Algorithm 1 details this process. For each interval $[x_k, x_{k+1}-1]$, we determine a sealing probability $p$ (Line 5). To ensure numerical stability, we handle the boundary cases $p \in \{0, 1\}$ such that $p = 0$ results in $flag = 0$ (no boundary), while $p = 1$ sets $idx = x_k$ with $flag = 1$ (immediate boundary). For the case $0 < p < 1$, we employ inverse transform sampling to compute the candidate as $x_k + \lceil \ln(u)/\ln(1-p) \rceil - 1$, with $u \sim U(0, 1)$. If it lies in this interval, we set $flag$ to 1, and to 0 otherwise (Lines 6–8). Upon deciding a boundary, *count* is reset to track next key (Line 9). After sampling, we run FOCompact to output the true boundaries (Lines 10–13).

**Algorithm 1:** DO Private Partition with $(\varepsilon, \delta)$-DP boundaries

**Input:** A sorted list of distinct keys $\mathcal{X} = [x_1, \ldots, x_n]$, with $x_k \in [1, D]$

**Output:** A sorted list of boundaries $\mathcal{S} = [s_1, \ldots, s_m]$, with $s_j \in [1, D]$; a list $\mathcal{C}$ with $c_j$ the $j$-th segment size; and maximum segment size $Z$

1 **Function** DOPartition($\mathcal{X}, \varepsilon, \delta$):
2 Init a list $\mathcal{I}_{\text{seal}}$ to store sampled candidate boundaries and indicators
3 $\zeta \leftarrow 2\ln(2D(2e^{\varepsilon} + \delta)/\delta)/\varepsilon$, $Z \leftarrow 5\zeta$, $x_{n+1} \leftarrow D$, $count \leftarrow 0$
4 **for** $k \leftarrow 1$ **to** $n$ **do**
5 $count \leftarrow count + 1$, $p \leftarrow \Pr[count + \text{Lap}_{\zeta}(2/\varepsilon) \geq 3\zeta + \text{Lap}_{\zeta}(2/\varepsilon)]$
6 $val \leftarrow (p = 0 \vee p = 1) \times 0.5 + (0 < p < 1) \times (1 - p)$
7 $idx \leftarrow x_k + (\lceil \ln(U(0, 1))/\ln(val) \rceil - 1) \times (0 < p < 1)$
8 $flag \leftarrow (p > 0) \wedge (idx < x_{k+1})$
9 Append $(idx, count, flag)$ to $\mathcal{I}_{\text{seal}}$, $count \leftarrow (\neg flag) \times count$
10 Run FOCompact($\mathcal{I}_{\text{seal}}$) to retain entries with $flag = 1$
11 Move the $idx$ field in $\mathcal{I}_{\text{seal}}$ to list $\mathcal{S}$ and the $count$ field in $\mathcal{I}_{\text{seal}}$ to list $\mathcal{C}$
12 Add $count$ to list $\mathcal{C}$ for the last unsealed segment
13 **return** $\mathcal{S}, \mathcal{C}, Z$ *// Note that $\mathcal{S}$ is released, whereas $Z$ is public*

*Example 5.3.* Figure 3 shows DOPartition on eight sorted keys over $[1, 50]$. Arrows indicate candidate samplings between adjacent keys, with orange ones yielding segment boundaries ($flag = 1$). For example, $(10, 2, 1)$ indicates a boundary at index 10 after two keys (i.e., segment size is 2), after which the counter resets. Overall, three boundaries partition the domain into four segments.

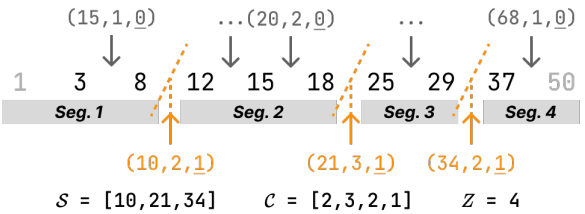



Fig. 3. Running example of DOPartition on domain $[1, 50]$

While DOPartition hides access patterns, its use of truncated Laplace noise alters the original Private Partition guarantees in two ways. First, truncation restricts the output space, changing the privacy guarantee of released boundaries and requiring a different threshold. Second, limiting noise magnitude strictly bounds non-final segment sizes in $[Z/5, Z]$, providing a stronger practical utility guarantee than the original. We formalize these privacy and utility guarantees below; see the full version [44] for proof details.

Theorem 5.4. DOPartition *releases* $(\varepsilon, \delta)$*-DP boundaries, satisfies* $(\varepsilon, \delta)$*-DO, and produces non-final segments of size in* $[Z/5, Z]$.

Proof Sketch. By interpreting truncated Laplace sampling as rejection sampling, DOPartition yields the same boundary distribution as the original Private Partition mechanism conditioned on a bounded-noise event. For an $\varepsilon$-DP Private Partition where this event occurs w.p. at least $1-\delta'$, the same-threshold DOPartition releases $(\varepsilon, 2\delta' e^{\varepsilon}/(1-\delta'))$-DP boundaries. Setting $\delta' = \delta/(2e^{\varepsilon}+\delta)$ and accounting for the induced distance-2 neighboring relation via group privacy together yield $(\varepsilon, \delta)$-DP boundaries for DOPartition under threshold $3\zeta = 6\ln(2D(2e^{\varepsilon}+\delta)/\delta)/\varepsilon$, while Lemma 5.2 implies the resulting non-final segment sizes fall in $[Z/5, Z]$, where $Z = 5\zeta$. Finally, since DOPartition samples once per interval and FOCompact is FO, its execution is a post-processing of public parameters and the released DP boundaries, thus establishing $(\varepsilon, \delta)$-DO. □

Regarding efficiency, DOPartition incurs $O(n \log n)$ computation and $O(\mathrm{Sort}(n))$ I/O, primarily due to the cost of FOCompact.

**Remark.** We extend the algorithm for iterative partitioning and dummy-item handling, and use DOPartition$^*(\mathcal{X}, \mathit{count}, c_{\text{full}}, \varepsilon, \delta)$ to denote it. Here, *count* carries over the previous state, while $c_{\text{full}}$ determines whether sampling continues to the domain endpoint $D$. To preserve obliviousness, dummy items (written as $\perp$) undergo the same sampling but always produce $flag = 0$, while intervals between consecutive real items are bridged for continuity. These modifications preserve the sampling distribution over real-item intervals, making successive calls equivalent to pausing and resuming one execution of DOPartition. Thus, DOPartition$^*$ inherits the guarantees of Theorem 5.4 with the same asymptotic complexity.

### 5.3 DO Item Assignment with Padding

**Algorithm 2:** $(\varepsilon, \delta)$-DO Item Assignment with Padding

**Input:** A list $\mathcal{X}$ of $n$ items sorted by keys; a sorted list $\mathcal{S}$ of $m$ boundaries from $\mathcal{X}$'s keys; a list of item counts $\mathcal{C}$ for $\mathcal{S}$; and a padding size $Z$

**Output:** A map $\mathcal{P}$ that maps $s_j$ to the list of items (including padding) in the segment ending at $s_j$

**Function** DOItemAssign$(\mathcal{X}, \mathcal{S}, \mathcal{C}, Z, \varepsilon, \delta)$**:**
- Init an empty working buffer $\mathcal{B}$ (with entries $b_j$)
- Obtain $\varepsilon$-DP prefix sums $\tilde{\mathcal{F}} = [\tilde{f}_1, \ldots, \tilde{f}_{m+1}]$ from $\mathcal{C}$ by BM
- $\eta \leftarrow \alpha_{\text{BM}}(m+1, \varepsilon/2, \delta)$, $w \leftarrow \lceil \eta/Z \rceil$, $s_{m+1} \leftarrow D$
- **for** $i \leftarrow 1$ **to** $\lceil (m+1)/w \rceil$ **do**
  - Read $\mathcal{X}$ up to $\min\{\tilde{f}_{\min\{iw, m+1\}} + \eta, n\}$-th item and add into $\mathcal{B}$
  - Run FOCompact$(\mathcal{B})$ to move all residual $\perp$ items in $\mathcal{B}$ to the end
  - $k \leftarrow (i-1)w+1$, $d \leftarrow 1$; init a mapping vector $\mathcal{V}$ with size $|\mathcal{B}|$
  - **for** $j \leftarrow 1$ **to** $|\mathcal{B}|-1$ **do**
    - $\mathcal{V}[b_j] \leftarrow (k-(i-1)w-1)\cdot Z + d$
    - $c_{\text{nxt}} \leftarrow (k \le iw) \wedge (b_j, b_{j+1} \neq \perp) \wedge (b_j.\text{key} \le s_k < b_{j+1}.\text{key})$
    - $k \leftarrow k + c_{\text{nxt}}$, $d \leftarrow (\neg\, c_{\text{nxt}}) \times d + 1$
  - $\mathcal{V}[b_{|\mathcal{B}|}] \leftarrow (k-(i-1)w-1)\cdot Z + d$
  - $\mathcal{B} \leftarrow$ FODistribute$(\mathcal{B}, \mathcal{V}, wZ + 4\eta)$, and fill empty slots with $\perp$
  - **for** $j \leftarrow (i-1)w+1$ **to** $\min\{iw, m+1\}$ **do**
    - $\mathcal{P}[s_j] \leftarrow [b_{j'Z+1}, \ldots, b_{(j'+1)Z}]$, with $j' = j-(i-1)\cdot w-1$
  - Remove the first $wZ$ items from $\mathcal{B}$ and truncate it to size $2\eta$
- **return** $\mathcal{P}$

After partitioning, items are assigned to segments for subsequent retrieval. To ensure DP compliance, however, the exact count of real items per segment must be concealed. To this end, we propose a DO item assignment algorithm that employs DP prefix sums for size obfuscation and pads each segment to a maximum size $Z$. Here, segment boundaries are public while keys remain private, meaning neighboring inputs share identical boundaries but differ by at most two underlying keys (from a single write). This enables seamless composition with `DOPartition`, which outputs these boundaries as function leakage. Algorithm 2 details this assignment procedure.

Specifically, we derive DP prefix sum list $\tilde{\mathcal{F}}$ (true list: $\mathcal{F}$) from item counts $\mathcal{C}$ (Line 3). The neighbor definition implies $\ell_1$-sensitivity 2 on $\mathcal{C}$, which requires calibrating the noise with $\varepsilon/2$. This establishes an error bound $\eta \leftarrow \alpha_{\mathrm{BM}}(m+1, \varepsilon/2, \delta)$ (Line 4), ensuring that reading up to index $\tilde{f}_i + \eta$ covers the first $i$ segments w.p. $\geq 1-\delta$. We charge this failure probability $\delta$ to the privacy budget and permit a non-oblivious fallback that guarantees correctness upon failure.

Next, we perform a batch assignment within an in-enclave working buffer $\mathcal{B}$, processing $w \leftarrow \lceil \eta/Z \rceil$ segments per iteration (Lines 4–5) by reading up to the $(\tilde{f}_{\min\{i+w, m+1\}} + \eta)$-th item (Line 6). Since at each step a fixed $O(\eta)$ extra items are read into $\mathcal{B}$, this batch strategy maintains the same asymptotic cost as processing one segment in the buffer but greatly reduces iterations. In each iteration, we apply `FOCompact` to move residual dummy items to the end, followed by a scan to decide each item's output position based on its segment index and offset (Lines 7–13). We then perform `FODistribute` to place items into their assigned positions and fill vacancies with $\perp$ (Line 14). This yields an output where the first $wZ$ items constitute $w$ segments, each padded to maximum size $Z$ (Lines 15–16).

*Example 5.5.* Figure 4 shows `DOItemAssign` procedure. For clarity, only keys are shown. Since $w = \lceil \eta/Z \rceil = 2$, each iteration processes two segments, so the procedure takes two iterations. In the first iteration, the algorithm reads the first six items (blue) according to the DP prefix sum $\tilde{\mathcal{F}}$ and estimation error $\eta$, determines their target segment indices and offsets $(k, d)$, and applies `FODistribute` to produce the first two padded segments. The item with key 25 belongs to the third segment and is placed at $wZ + 1$ for the next iteration. In the second pass, the algorithm reads the remaining items, including the last two keys (red), and processes them with the carried-over item to produce the third and fourth padded segments.

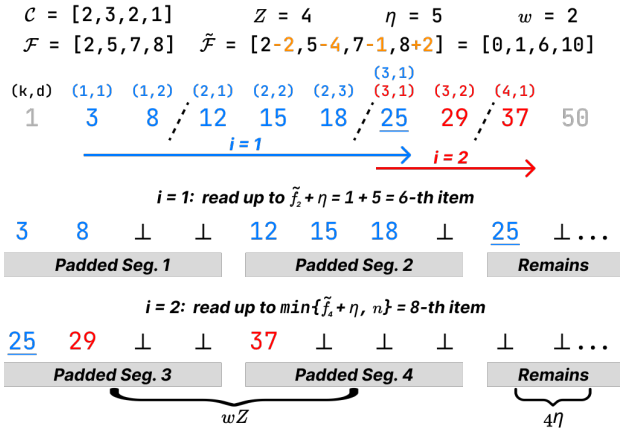



Fig. 4. Running example (continued): `DOItemAssign`.

We now formalize the assignment procedure's privacy guarantee.

Lemma 5.6. `DOItemAssign` *satisfies* $(\varepsilon, \delta)$*-DO.*

Proof Sketch. Since the item-count list $\mathcal{C}$ has $\ell_1$-sensitivity at most 2, the BM calibrated with $\varepsilon/2$ releases $\varepsilon$-DP prefix sums. As `FOCompact` and `FODistribute` are FO, the remaining execution depends only on these prefix sums and public parameters. A non-oblivious fallback is invoked only when DP prefix sums exceed their error bounds, ensuring correctness; this event occurs w.p. at most $\delta$ and is charged to the approximate privacy term, yielding $(\varepsilon, \delta)$-DO. □

We next justify that the assignment procedure requires a buffer of size at most $wZ + 4\eta$. On one hand, after processing $w$ segments in each step, we safely truncate the buffer to size $2\eta$ (Line 17), as at most $2\eta$ items are over-read. In the next iteration, we read up to the $(\tilde{f}_{(i+1)w}+\eta)$-th item. Since $f_{iw}$ items have been consumed, this introduces at most $wZ+2\eta$ fresh items. Together with the remaining $2\eta$ items, the buffer peaks at $wZ+4\eta$. On the other hand, we set the output array size of `FODistribute` to $wZ+4\eta$ (Line 14). This size is sufficient because the first $wZ$ slots accommodate items from the $w$ processed segments, followed by at most $2\eta$ over-read items and $2\eta$ dummy items, placed starting from index $wZ+1$ (or earlier).

We then analyze expected cost of `DOItemAssign`. The BM for DP prefix sums takes $O(m)$ computation [9]. Given $n$ items, there are $m = \Theta(n/Z)$ boundaries, yielding $\Theta(n/\max\{\eta, Z\})$ iterations. Each iteration is dominated by `FOCompact` and `FODistribute`, incurring $O(|\mathcal{B}| \log |\mathcal{B}|)$ computation. Since $|\mathcal{B}| \le wZ + 4\eta = O(\max\{\eta, Z\})$, the total computation is $O(n \log \max\{\eta, Z\})$. With $\varepsilon = \Theta(1)$ and $\delta = O(n^{-c})$ for a constant $c > 1$, this simplifies to $O(n \log \log D)$. In terms of I/O, the BM can be adapted to achieve $O(m/B)$ I/Os for a static dataset. The main loop then sequentially reads the $n$ items of $\mathcal{X}$ into the enclave and writes out $(m+1)Z = \Theta(n)$ items. Consequently, the total I/O complexity of `DOItemAssign` is $O(n/B)$.

### 5.4 DO Merge of Partition-Based PDSs

Thus far, we have built the partition-based PDS with segments padded to size $Z$. To integrate it into our LSM-inspired structure, we need an efficient DO merge combining two PDSs. Here, neighboring inputs share identical boundaries in both PDSs, but differ in exactly one of them by at most two underlying keys (from a single write).

Algorithm 3 outlines the merge process. Linear-cost two-pointer `MergeSorted` merges sorted boundary lists $\mathcal{S}_1$ and $\mathcal{S}_2$ into sorted $\mathcal{S} = [s_1, \ldots, s_{p+q}]$ (Line 2). W.l.o.g., we assume $\mathcal{S}_1$ and $\mathcal{S}_2$ are derived from the same public parameters $D$, $\varepsilon$, $\delta$, and hence $Z$. The algorithm then runs for $p + q + 1$ iterations (Lines 5–20), where iteration $k$ processes all real items up to $s_k$, with $s_{p+q+1} \leftarrow D$.

In iteration $k$, the buffer has capacity $3Z+2$. We orchestrate the read sequence (Lines 4 and 15–20) to control its layout: the first $Z+1$ slots contain $x_{last}$ from the previous iteration, if present, and $Z$ remaining or newly fetched items with key $\le s_k$ from a PDS segment ending at $s_k$; the next $Z+1$ slots are dummy items ($\perp^{Z+1}$); and the last $Z$ slots contain items from a segment of the other PDS. In the first and last parts, real items may be followed by $\perp$ items.

Next, we scan the buffer to count the output real items for this iteration (Line 6). To hide this count, we first pad the real items to size $2Z+2$ by converting requisite $\perp$ items into fillers $\perp_{s_k}$ with key $= s_k$ and ver $= 0$, where ver indicates data freshness (Lines 7–9). By treating $\perp$ keys as $+\infty$, the first $2Z+2$ and last $Z$ slots remain sorted by increasing keys and decreasing versions, as new fillers naturally rank between the real and $\perp$ items. Reversing the first $2Z+2$ slots then creates a bitonic sequence, efficiently sorted via `BitonicMerge`, a standard FO merging network (Line 10).

After merging, the first $2Z+1$ slots contain real items (key $\le s_k$) and fillers. We deduplicate them by retaining only the maximum-ver item per key via a sequential oblivious scan followed by `FOCompact`, and then invoke `DOPartition`* to obtain and record partial boundaries. Meanwhile, the last $Z$ slots hold the remaining real items followed by $\perp$ items. Notably, we set $x_{\text{last}}$ as the

**Algorithm 3:** DO Merge of Partition-Based PDSs with $(\varepsilon, \delta)$-DP boundaries

**Input:** Two sorted boundary lists $\mathcal{S}_1 = [s_{1,1}, \ldots, s_{1,p}]$, $\mathcal{S}_2 = [s_{2,1}, \ldots, s_{2,q}]$ for two PDSs; maps $\mathcal{P}_1, \mathcal{P}_2$ linking each segment to its items; a padding size $Z$; and an upper limit $N_{\text{real}}$ on real item count

**Output:** A sorted list $\hat{\mathcal{X}}$ of merged items end-padded with $\perp$ items; a sorted list of boundaries $\hat{\mathcal{S}}$; a count list $\hat{\mathcal{C}}$ and maximum segment size $\hat{Z}$

1 **Function** DOPMerge$(\mathcal{S}_1, \mathcal{S}_2, \mathcal{P}_1, \mathcal{P}_2, Z, N_{real}, \varepsilon, \delta)$:
2 Init buffer $\mathcal{B}$ (entries $b_j$, capacity $3Z+2$), $\mathcal{S} \leftarrow$ MergeSorted$(\mathcal{S}_1, \mathcal{S}_2)$
3 $s_{1,p+1} = s_{2,q+1} = s_{p+q+1} \leftarrow D,\ i_1 = i_2 \leftarrow 1,\ cnt_{\text{seg}} \leftarrow 0,\ count \leftarrow 0$
4 **if** $s_1 \in \mathcal{S}_1$ **then** add $\mathcal{P}_1[s_{1,i_1}]$, $\perp^{Z+2}$, $\mathcal{P}_2[s_{2,i_2}]$ to $\mathcal{B}$ **else** Vice Versa
5 **for** $k \leftarrow 1$ **to** $p+q+1$ **do**
6 **for** $j \leftarrow 1$ **to** $|\mathcal{B}|$ **do** $cnt_{\text{seg}} \leftarrow cnt_{\text{seg}} + (b_j \neq \perp) \wedge (b_j.\text{key} \leq s_k)$
7 **for** $j \leftarrow 1$ **to** $2Z+2$ **do**
8 $c_{\text{fill}} \leftarrow (b_j = \perp) \wedge (cnt_{\text{seg}} < 2Z+2),\ \perp_{s_k} \leftarrow \perp(\text{key } s_k, \text{ver } 0)$
9 $b_j \leftarrow c_{\text{fill}} \times \perp_{s_k} + (\neg\, c_{\text{fill}}) \times b_j,\ cnt_{\text{seg}} \leftarrow cnt_{\text{seg}} + c_{\text{fill}}$
10 Reverse $\mathcal{B}[1{:}2Z+2]$, then BitonicMerge$(\mathcal{B}, \langle \mathcal{B}.\text{key}\uparrow, \mathcal{B}.\text{ver}\downarrow\rangle)$
11 Scan $\mathcal{B}[1{:}2Z+1]$ to keep max-ver item per key (mark rest $\perp$ + FOCompact), set $x_{\text{last}}$ as the last real item, then let $b_{2Z+2} \leftarrow x_{\text{last}}$
12 $\mathcal{S}', \mathcal{C}', \hat{Z} \leftarrow$ DOPartition$^*(\mathcal{B}[1{:}2Z+1], count, k = p+q+1, \varepsilon, \delta)$
13 Append $\mathcal{S}'$ to $\hat{\mathcal{S}}$; Pop $\mathcal{C}'$ into $count$ and then append $\mathcal{C}'$ to $\hat{\mathcal{C}}$
14 **if** $k = 1$ **then** append $\mathcal{B}[1{:}2Z]$ to $\hat{\mathcal{X}}$ **else** append $\mathcal{B}[2{:}2Z+1]$ to $\hat{\mathcal{X}}$
15 **if** $s_k \in \mathcal{S}_e$ *for* $e \in \{1, 2\}$ **then** $i_e \leftarrow i_e + 1$
16 **if** $(k \neq p+q+1) \wedge (s_k, s_{k+1} \in same\ \mathcal{S}_e)$ **then**
17 $\mathcal{B}[1{:}Z+1] \leftarrow b_{2Z+2} \parallel \mathcal{P}_e[s_{e,i_e}]$ // $\parallel$ *denotes concatenation*
18 **else if** $(k \neq p+q+1) \wedge (s_k, s_{k+1} \notin same\ \mathcal{S}_e)$ **then**
19 $\mathcal{B}[1{:}Z+1] \leftarrow \mathcal{B}[2Z+2{:}3Z+2],\ \mathcal{B}[2Z+3{:}3Z+2] \leftarrow \mathcal{P}_e[s_{e,i_e}]$
20 $\mathcal{B}[Z+2{:}2Z+2] \leftarrow \perp^{Z+1},\ cnt_{\text{seg}} \leftarrow 0$
21 Append $count$ to $\hat{\mathcal{C}}$; run DOCompact$(\hat{\mathcal{X}})$ and then retain $N_{\text{real}}$ items
22 **return** $\hat{\mathcal{X}}, \hat{\mathcal{S}}, \hat{\mathcal{C}}, \hat{Z}$

maximum real item with key $\leq s_k$ and pass it to slot $2Z+2$ for the next iteration, since a sample exists between $x_{\text{last}}$ and the first item with key $> s_k$ (Lines 11–13). We then append $2Z$ items from buffer to $\hat{\mathcal{X}}$, possibly including $\perp$ items, as $2Z$ bounds the real items between adjacent merged boundaries (Line 14). To prepare for the next iteration, we replenish buffer with $Z+1$ $\perp$ items and subsequent segment of the exhausted PDS (Lines 15–20), keeping aforementioned data layout.

Finally, upon collecting all real items, we run DOCompact to move all $\perp$ items to the end and truncate $\hat{\mathcal{X}}$ to size $N_{\text{real}}$ (Line 21), preventing leakage arising from deduplication. Here, $N_{\text{real}}$ is a given upper limit on the total number of real items, bounded by $(p+q+2) \cdot Z$.

*Example 5.7.* Figure 5 illustrates merging constructed PDS 1 with new PDS 2, both containing four padded segments. Circled numbers show the segment read order. The lower half traces working buffer $\mathcal{B}$ in iteration $k = 1$. As the first merged boundary is $s_1 = 7 \in \mathcal{S}_2$, the buffer loads $\mathcal{P}_2[s_{2,1}]$ then $\mathcal{P}_1[s_{1,1}]$. To prepare for BitonicMerge, the algorithm marks $2Z+2-cnt_{\text{seg}} = 6$ dummy items with $c_{\text{fill}} = 1$ and converts them to fillers $\perp_7$, so that reversing the first $2Z+2$ positions makes the sequence bitonic. After merging, the largest item with key at most 7 (here, key 6) is retained as the carryover item $x_{\text{last}}$ and placed at position $2Z+2$ for the next iteration (deduplication is omitted here). Running DOPartition$^*$ on the first $2Z+1$ items outputs a new boundary at 4 after two real keys, leaving an inherited $count$ of 1 (for key 5). Since $k = 1$, the first $2Z$ items are appended to $\hat{\mathcal{X}}$.

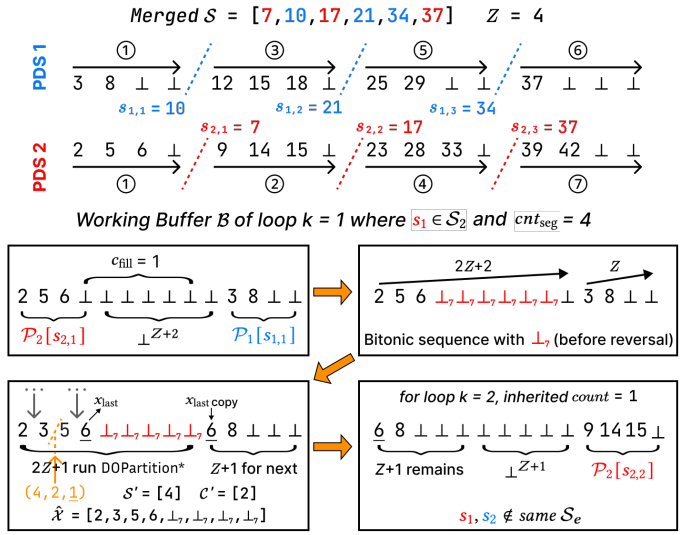



Fig. 5. Running example (continued): DOPMerge.

Then, the $2Z+1$ processed items are discarded, while the remaining $Z+1$ items stay buffered. Finally, since the current segment from PDS 2 has been exhausted, the buffer fetches $\mathcal{P}_2[s_{2,2}]$, restoring the required layout for iteration $k=2$.

We now formalize the merge procedure's privacy guarantee.

Lemma 5.8. DOPMerge *releases* $(\varepsilon, \delta)$*-DP boundaries and is DO.*

Proof Sketch. By Theorem 5.4, the calls to DOPartition* release $(\varepsilon, \delta)$-DP boundaries and satisfy DO. Aside from these calls and the final DOCompact, the execution is FO or determined by public parameters and the released boundaries. For neighboring inputs yielding identical boundaries, the real/dummy patterns in $\hat{\mathcal{X}}$ have Hamming distance at most 2 and satisfy the neighboring relation required by DOCompact. Adaptive composition of these calls with DOCompact establishes DO. □

We finally analyze the complexity of DOPMerge, which merges $n$ real items from two PDSs over $p+q+1=\Theta(n/Z)$ iterations. With a buffer size $3Z+2$, each iteration is dominated by BitonicMerge, FOCompact, and DOPartition*, costing $O(Z \log Z)$ computation. Over all iterations, together with the final DOCompact, the computation is $O(n \log Z) = O(n \log \log D)$. Streaming both PDSs into the enclave, sequentially writing an $O(n)$ output, and running the final DOCompact incur $O(n/B)$ I/Os.

## 6 SapphMap Design

In this section, we describe the design of SapphMap. Building on the LSM-inspired architecture in Section 2 and DO algorithms in Section 5, we first discuss write operations in Section 6.1 and basic query processing in Section 6.2, presenting workflows and analyses. We further describe a theoretical refinement using fractional cascading in Section 6.3 to improve asymptotic query efficiency. Finally, we analyze system parameter constraints in Section 6.4 to derive block-size and enclave-memory requirements for feasibility.

### 6.1 Write Operations

SapphMap supports KV writes (i.e., insertions, updates, and deletions) by unifying them as insertion of an item $\langle$op, key, value, ver$\rangle$. Here, op $\in$ {**ins**, **del**} suffices for new- and existing-key operations, while ver is a monotonically increasing version counter (e.g., a timestamp) that preserves ordering.

*6.1.1 Bulk loading.* Given an initial dataset, we employ bulk loading to construct the initial state of SapphMap, avoiding sequential insertion overhead. Consistent with the unified write format, each initial item is treated as an insertion (op = **ins**) with base version (ver = 0). If dataset fits within enclave memory, it is stored in buffer $\mathcal{T}_0$. Otherwise, we first apply `FOSort` to sort items by key. Next, we invoke `DOPartition` and `DOItemAssign` to construct the PDS. Finally, this PDS is placed at the shallowest level $\mathcal{T}_k$ whose nominal capacity $\gamma^k M$ (i.e., number of input real items) can accommodate the dataset size $N$. Notably, each level physically contains more items due to padding, but the total remains asymptotically equivalent.

In the bulk loading phase, each item incurs an amortized cost of $O(\log N + \log\log D)$ in computation and $O(\mathrm{Sort}(N)/N)$ in I/O complexity. Importantly, these bounds already account for the overhead from privacy budget allocation (see Section 6.1.5 for details).

**Algorithm 4:** Write operation on a $(\varepsilon_{\text{tot}}, \delta_{\text{tot}})$-DO SapphMap

**Input:** A write operation $x = \langle \text{op}, \text{key}, \text{value}, \text{ver}\rangle$; and a SapphMap $\mathbb{T}$
**Output:** The updated SapphMap $\mathbb{T}$ with operation $x$ applied

1 **Function** `DOWrite`$(x, \mathbb{T}, \varepsilon_{\text{tot}}, \delta_{\text{tot}})$**:**
2 $\quad \mu \leftarrow (\gamma-1)\cdot\lceil \log_\gamma(N_{\max}/M)\rceil + 1,\ \varepsilon \leftarrow \varepsilon_{\text{tot}}/(3\mu),\ \delta \leftarrow \delta_{\text{tot}}/(3\mu)$
3 $\quad$ Append the write operation $x$ to the in-enclave buffer $\mathcal{T}_0$
4 $\quad$ **if** $|\mathcal{T}_0| = M$ **then** `DOFlush`$(\mathbb{T}, \varepsilon, \delta)$
5 **Function** `DOFlush`$(\mathbb{T}, \varepsilon, \delta)$**:**
6 $\quad \mathcal{X}_0 \leftarrow$ `FOSort`$(\mathcal{T}_0, \langle \mathcal{T}_0.\text{key}\uparrow, \mathcal{T}_0.\text{ver}\downarrow\rangle),\ i \leftarrow 0$
7 $\quad$ Scan $\mathcal{X}_0$ to keep max-ver item per key (mark others as $\perp$ + `FOCompact`)
8 $\quad \mathcal{S}_0, \mathcal{C}_0, Z \leftarrow$ `DOPartition`$^*(\mathcal{X}_0.\text{key}, 0, \text{true}, \varepsilon, \delta)$ *// Same $Z$ throughout*
9 $\quad \mathcal{P}_0 \leftarrow$ `DOItemAssign`$(\mathcal{X}_0, \mathcal{S}_0, \mathcal{C}_0, Z, \varepsilon, \delta),\ \mathcal{T}_0 \leftarrow (\mathcal{S}_0, \mathcal{P}_0)$
10 $\quad$ **while true do**
11 $\quad\quad$ **if** $\mathcal{T}_{i+1} = \emptyset$ **then**
12 $\quad\quad\quad \mathcal{T}_{i+1} \leftarrow \mathcal{T}_i,\ \mathcal{T}_{i+1}.\text{size} \leftarrow 1,\ \mathcal{T}_i \leftarrow \emptyset,\ \mathcal{T}_i.\text{size} \leftarrow 0$
13 $\quad\quad\quad \mathbb{T}.\text{depth} \leftarrow \max(i+1, \mathbb{T}.\text{depth})$; **break**
14 $\quad\quad (\mathcal{S}_{i+1}, \mathcal{P}_{i+1}) \leftarrow \mathcal{T}_{i+1},\ (\mathcal{S}_i, \mathcal{P}_i) \leftarrow \mathcal{T}_i,\ N_{\text{real}} \leftarrow (\mathcal{T}_{i+1}.\text{size} + 1)\cdot\gamma^i M$
15 $\quad\quad$ *// Drop* op = **del** *at bottom* `DOPMerge`*; give $(\varepsilon, \delta)$ to inner* `DOCompact`
16 $\quad\quad \mathcal{X}_{i+1}, \mathcal{S}_{i+1}, \mathcal{C}_{i+1}, _ \leftarrow$ `DOPMerge`$(\mathcal{S}_i, \mathcal{S}_{i+1}, \mathcal{P}_i, \mathcal{P}_{i+1}, Z, N_{\text{real}}, \varepsilon, \delta)$
17 $\quad\quad \mathcal{P}_{i+1} \leftarrow$ `DOItemAssign`$(\mathcal{X}_{i+1}, \mathcal{S}_{i+1}, \mathcal{C}_{i+1}, Z, \varepsilon, \delta)$
18 $\quad\quad \mathcal{T}_{i+1} \leftarrow (\mathcal{S}_{i+1}, \mathcal{P}_{i+1}),\ \mathcal{T}_{i+1}.\text{size}\,{+}{+},\ \mathcal{T}_i \leftarrow \emptyset,\ \mathcal{T}_i.\text{size} \leftarrow 0$
19 $\quad\quad$ **if** $\mathcal{T}_{i+1}.\text{size} \geq \gamma$ **then** $i \leftarrow i+1$ **else break**
20 $\quad$ Construct B+-trees over new boundaries for every level from $\mathcal{T}_1$ to $\mathcal{T}_{i+1}$

*6.1.2 Workflow for Dynamic Writes.* Algorithm 4 details dynamic writes. Incoming operations append to the in-enclave buffer $\mathcal{T}_0$ of size $M$ (Line 3). When full, `DOFlush` sorts the buffer, deduplicates by retaining only maximum-ver item per key via a sequential oblivious scan and `FOCompact`, and constructs the initial PDS (Lines 6–9). This PDS enters the hierarchy, where each level $\mathcal{T}$ contains boundaries $\mathcal{S}$ and map $\mathcal{P}$. A `while` loop (Lines 10–19) manages cascading merges: if the target $\mathcal{T}_{i+1}$ is empty, the PDS migrates directly (Lines 11–13); otherwise, it merges with the resident PDS to form a new partitioned PDS (Lines 14–18). Such a merge inherently retains only the maximum-ver item per key, and at the bottom level also discards op = **del** items, providing garbage collection to control space. Finally, the loop continues if the resulting level reaches its logical capacity $\gamma$ (Line 19). For efficient query processing, we maintain a per-level B+-tree over the released DP boundaries

to accelerate overlapping-segment searches, which is regenerated upon PDS construction (Line 20). Its persistent representation resides in untrusted storage. Structurally, the size of each level is the number of PDSs merged from the preceding level, not item count. The fixed logical capacity $\gamma$ bounds total DO invocations under maximum writes $N_{\max}$, thus enabling privacy budget allocation.

*6.1.3 Privacy Budget Allocation.* We outline the allocation of the total privacy budget $(\varepsilon_{\text{tot}}, \delta_{\text{tot}})$ by tracking the privacy loss of a single item. For a fixed item, algorithms on disjoint substructures incur no additional privacy loss under parallel composition; hence, it suffices to account only for the DO algorithms involving it. Then, by sequential composition, its total privacy cost is the sum of budgets consumed by these algorithms along its path through the levels.

To bound cumulative cost, we assume at most $N_{\max}$ write operations, counting both bulk-loaded and dynamic writes. This bounds the SapphMap height by $\lfloor \log_\gamma(N_{\max}/M) \rfloor + 1$, which constrains the worst-case composition path and enables equal per-level budgets. For unbounded writes, one could employ a convergent allocation (e.g., $\frac{6}{\pi^2(i+1)^2}$ fraction to $\mathcal{T}_i$), but this is beyond our scope.

The PDS constructions are confined to $\lceil \log_\gamma(N_{\max}/M) \rceil$ levels. Within each level, a single item participates in at most $\gamma$–1 constructions. Accounting for the initial PDS construction (in $\mathcal{T}_0$ or during bulk loading), an item undergoes at most $\mu = (\gamma - 1) \cdot \lceil \log_\gamma(N_{\max}/M) \rceil + 1$ PDS constructions.

Concretely, each construction incurs privacy costs from at most three operations: (1) determining DP boundaries in `DOPartition` or `DOPMerge`; (2) computing the DP prefix sums in `DOItemAssign`; and (3) executing the internal `DOCompact` in `DOPMerge`. Hence, given the construction bound $\mu$, we allocate three budget units to each construction to cover the worst-case execution path, resulting in an atomic budget of $\varepsilon = \varepsilon_{\text{tot}}/(3\mu)$ (and similarly for $\delta$) (Line 2).

*6.1.4 Obliviousness Analysis.* We now combine the prior component guarantees with the privacy-budget allocation in Section 6.1.3 to establish the end-to-end DO guarantee of SapphMap.

Theorem 6.1. *For any write sequence of length at most $N_{\max}$, SapphMap is $(\varepsilon_{\text{tot}}, \delta_{\text{tot}})$-doubly-DO.*

Proof Sketch. It suffices to analyze `DOFlush`, as `DOWrite` simply appends to the in-enclave buffer and flushes at its public capacity $M$. `DOFlush` comprises `DOPartition`*, `DOItemAssign`, `DOPMerge`, and leakage-free FO steps. By Theorem 5.4 and Lemmas 5.6 and 5.8, these three components' executions satisfy DP whether observed via enclave computation (e.g., working buffer operations) or untrusted-storage I/O (e.g., enclave-buffer transfers). Crucially, their intermediate outputs w.h.p. preserve bounded-distance neighboring relations required for adaptive composition.

By Section 6.1.3, parallel and sequential composition bound the cost by $(3\mu\varepsilon, 3\mu\delta) = (\varepsilon_{\text{tot}}, \delta_{\text{tot}})$, ensuring doubly-DO privacy. □

*6.1.5 Complexity Analysis.* We next analyze the amortized cost of a dynamic write to SapphMap over $N$ operations, first examining how each DO algorithm's cost (on input size $n$) varies with privacy budget allocation (Section 6.1.3). For simplicity, we assume $\log N_{\max} = \Theta(\log D)$, and recall that $\varepsilon_{\text{tot}} = \Theta(1)$ with $\delta_{\text{tot}} = O(N_{\max}^{-c})$ for $c > 1$.

The allocation strategy governs the main loops of `DOItemAssign` and `DOPMerge`, as well as the internal `DOCompact` of the latter. It scales their in-enclave buffer footprint by a factor of $\approx 3\mu$. Since $\gamma$ is a small constant or $B^\rho$ with $\rho \in (0, 1)$ (see Section 6.4), this overhead is absorbed into logarithmic term, preserving $O(n \log\log D)$ computation. Moreover, I/O remains $O(n/B)$ independent of privacy budget, as it relies on sequential reads and writes of $O(n)$ items.

Next, we analyze the amortized cost per write. Within the in-enclave buffer $\mathcal{T}_0$, each item first undergoes `FOSort` and `FOCompact`, incurring an amortized $O(\log M)$ computation and zero I/O. Subsequently, `DOPartition`* and `DOItemAssign` construct a PDS to flush into the hierarchy, adding amortized $O(\log\log D)$ computation and $O(1/B)$ I/O. For level $\mathcal{T}_i$ with nominal capacity $\gamma^i M$, a

full cycle performs $\gamma-1$ PDS constructions (`DOPMerge` + `DOItemAssign`), each followed by a B+-tree regeneration. Due to the pre-sorted boundaries, this linear regeneration cost is subsumed by the construction. Thus, each write amortizes the cost of $(\gamma-1)/(\gamma^i M)$ constructions, yielding $O(\gamma \log\log D)$ computation and $O(\gamma/B)$ I/Os.

Across all levels, amortized per-write computation is $O(\log M + \gamma \log_\gamma(N/M) \log\log D)$, simplifying to $O(\gamma \log_\gamma N \log\log D)$. The amortized per-write I/O cost is $O((\gamma/B)\log_\gamma(N/M))$. Overall, this matches the I/O complexity of standard leveled LSM-trees, while incurring only a multiplicative $O(\log\log D)$ overhead in computation. Crucially, the tree height depends on the total operation count $N$ (rather than live KV pairs) to bound the PDS constructions for privacy budget allocation.

As suggested, bulk loading is more efficient than dynamic item-by-item writes, making it better suited for initializing SapphMap.

## 6.2 Query Processing

*6.2.1 Workflow.* SapphMap supports point and range queries. Unlike DO writes, queries do not protect their induced execution footprints, but strictly prevent additional leakage (e.g., intra-segment data distribution) beyond what PDS construction reveals. In particular, B+-tree traversal and its branching decisions only post-process released DP boundaries and thus incur no further leakage.

Due to the LSM-inspired hierarchy, multiple versions of the same key may coexist across levels. For point queries, SapphMap obliviously resolves inter-level duplicates and returns the latest matching KV pair or reports non-existence. For range queries, SapphMap follows prior work [9] by returning a superset containing all matching KV pairs alongside stale versions and dummy items, with the pruning task offloaded to the client. This approach effectively obfuscates the output volume to mitigate database reconstruction attacks [52], with negligible linear-time client overhead.

Algorithm 5 details the basic procedure for range queries. The process begins by appending items from buffer $\mathcal{T}_0$ to the candidate list $\mathcal{I}_{\text{out}}$ (Line 2). It then iterates through levels, using the B+-tree to locate the first overlapping segment (Line 5). From this index, the algorithm traverses all overlapping segments, appending their items to $\mathcal{I}_{\text{out}}$ (Lines 6–8). Point queries share this iterative segment lookup logic but track a single result variable. The algorithm instead scans the target segments to obliviously update the result upon finding a matching key with a higher version.

**Algorithm 5:** Range query on a DO SapphMap

**Input:** A query range $[l, r]$; and a SapphMap $\mathbb{T}$
**Output:** A candidate list $\mathcal{I}_{\text{out}}$ guaranteed to contain all KV pairs in $[l, r]$

```
1 Function RangeQuery(l, r, 𝕋):
2     I_out ← ∅; append all items in the in-enclave buffer T_0 to I_out
3     for i ← 1 to 𝕋.depth do
4         (S_i, P_i) ← T_i, S_i ← S_i ‖ D
5         Find index j of the first s_{i,j} ∈ S_i such that s_{i,j} ≥ l using B+-tree
6         while j ≤ |S_i| do
7             Append all items in P_i[s_{i,j}] to I_out
8             if s_{i,j} ≥ r then break else j ← j + 1
9     return I_out
```

*6.2.2 Complexity Analysis.* We study basic query cost of SapphMap. Here, we assume that each segment spans a constant number of I/O blocks (i.e., $B = \Omega(Z)$); see Section 6.4 for requirements.

Besides the linear scan of the in-enclave buffer, the query cost consists of the segment lookup overhead at each level and the sequential scanning of items within the identified segments. First,

the total segment lookup cost sums over all $O(\log_\gamma(N/M))$ levels, with each level $i$ involving a B+-tree search on $O(\gamma^i M)$ items. Regarding scanned items, let $R$ denote the total number of items in SapphMap matching the range $[l, r]$. At each level, at most two segments partially overlap with the query boundaries, potentially requiring a scan of up to $2Z$ non-matching items. For non-final segments fully contained within $[l, r]$, the dummy item count is proportional to the real items, bounding their contribution to $O(R)$. Thus, we scan $O(Z\log_\gamma(N/M)+R)$ items across all levels.

Overall, a basic range query incurs $O(Z\log_\gamma(N/M)+R+M)$ computation and an I/O cost of $O(\log_\gamma(N/M)\log_B N+R/B)$. Similarly, a basic point query accesses one segment per level. Its computation cost is $O(Z\log_\gamma(N/M)+M)$, and its I/O cost is $O(\log_\gamma(N/M)\log_B N)$.

The above bounds characterize the basic query procedure. A simple theoretical refinement is to store each segment in an ORAM. This ORAM-based optimization reduces the point-query computation to $O(\log_\gamma(N/M)\cdot(\log N+\log^2 Z)+M)$, where the $\log^2 Z$ term comes from intra-segment oblivious binary search. Crucially, this incurs no additional amortized write penalty, since ORAM initialization overhead is subsumed by the PDS construction at each level. We next present another theoretical refinement using fractional cascading to further eliminate repeated segment lookups across levels, thereby matching query I/O bounds of standard leveled LSM-trees.

### 6.3 Refinement via Fractional Cascading

*Fractional cascading* [13, 14] accelerates iterative searches across sorted sequences via cross-level pointers. Theoretically, it applies to SapphMap via bottom-up propagation to improve asymptotic query efficiency with linear-space overhead. We note that it is not implemented due to large constant factors. For each level $i$, we construct an augmented boundary list $\tilde{\mathcal{S}}_i$ by merging local boundaries $\mathcal{S}_i$ with samples from augmented boundaries of the level below, taken every $2\gamma$. During this merge, each sample retains its down pointer and a pointer to the preceding local boundary, while each local boundary retains pointers to the original boundary and preceding sample. These samples are materialized with augmented boundary updates to avoid random I/O during upper-level access. Thus, only the first level requires B+-tree construction and search, while deeper levels are navigated via down pointers to locate overlapping segments.

Queries start at the first level via B+-tree and cascade downwards. Locating target $u \in \tilde{\mathcal{S}}_i$ at level $\mathcal{T}_i$ yields both the local segment and descent path in $O(1)$ time: if $u$ is a sample (resp. local boundary), its preceding local boundary's successor (resp. $u$ itself) identifies the first overlapping segment, while the transition to level $\mathcal{T}_{i+1}$ follows its own (resp. preceding sample's) down pointer. This allows $u$'s position in $\mathcal{T}_{i+1}$ to be refined with at most a $2\gamma$-entry backward (resp. forward) scan.

Fractional cascading reduces query costs without extra asymptotic setup overhead. Notably, sampling every $2\gamma$ positions incurs at most a $2\times$ size blowup relative to the original boundaries, preserving the asymptotic space complexity. For structure updates, each $\mathcal{T}_i$ construction requires refreshing the fractional cascading structures across all upper levels ($\mathcal{T}_1 \ldots \mathcal{T}_i$). Hence, each write at $\mathcal{T}_i$ amortizes a $\gamma/(\gamma^i M)$ share of the update cost, including the first migrated PDS. Moreover, the total update overhead remains linear in $\mathcal{T}_i$'s data volume, as it dominates the aggregate size of all upper levels (for $\gamma \geq 2$). Thus, the added amortized per-write cost at $\mathcal{T}_i$ is $O(\gamma)$ in computation and $O(\gamma/B)$ in I/O; being subsumed by PDS constructions, it adds no asymptotic execution overhead.

For query I/O, segment lookup complexity reduces to a first-level search cost of $O(\log_B(\gamma M))$ plus a descent cost of $O(\log_\gamma(N/M))$. The descent cost arises because, when $2\gamma \ll B$, scanned entries per level occupy a constant number of consecutive I/O blocks. Moreover, since the first-level search cost depends only on fixed $M$ and $\gamma$, the descent cost (growing with $N/M$) typically dominates. Thus, point query I/O simplifies to $O(\log_\gamma(N/M))$, with range queries adding an $O(R/B)$ term. Incorporating the ORAM strategy, point query computation also reduces to $\tilde{O}(\log N + M)$.

### 6.4 System Parameter Constraints

The design of SapphMap is governed by key parameter constraints. First, to ensure DO algorithms function properly, enclave memory $M$ must accommodate the maximum working buffer required by `DOCompact`, `DOItemAssign`, and `DOPMerge`. Second, to bound query I/O cost, each segment must span a constant number of I/O blocks of size $B$. We also note that external `FOSort` requires the tall-cache assumption. Accordingly, we set the fan-out $\gamma = B^\rho$, where $\rho \in (0,1)$ is a fixed constant or chosen to yield constant $\gamma$ (e.g., $\rho = \log_B 2$ for $\gamma = 2$). This choice balances tree height against system constraints. For asymptotic simplicity, we abstract away constant factors, but they still warrant careful empirical evaluation in practice.

*6.4.1 Block Size.* For each DO algorithm, with the budget split over critical path $3\mu$, it follows that for $\gamma = B^\rho$, we have $1/\varepsilon = \Theta(\mu) = \Theta(\gamma \log_\gamma(N_{\max}/M)) = \Theta(B^\rho \log D/\log B)$, and $\ln(1/\delta) = \Theta(\log(\mu \cdot N_{\max}^c)) = \Theta(\log D)$. Substituting them into segment size $Z = \Theta((1/\varepsilon)\log(D/\delta))$, we derive $Z = \Theta(B^\rho \log^2 D/\log B)$. Similarly, for a constant $\gamma$, we have $1/\varepsilon = \Theta(\log D)$, yielding $Z = \Theta(\log^2 D)$.

The block size constraint $B = \Omega(Z)$ implies $B = \Omega(\log^2 D)$ for a constant $\gamma$. For $\gamma = B^\rho$, this becomes $B^{1-\rho}\log B = \Omega(\log^2 D)$, which yields the sufficient bound $B = \Omega((\log D)^{2/(1-\rho)})$ (ignoring $\log B$). In both cases, the derived bounds on $B$ naturally satisfy the requirement $B = \Omega(\log^2 N)$ imposed by sorting primitive `Flexway-Osort`.

*6.4.2 Enclave Memory Size.* The enclave memory $M$ is shared between the persistent buffer level ($\mathcal{T}_0$) in SapphMap and the transient working buffer ($\mathcal{B}$) utilized during PDS constructions. The per-level B+-tree storage is external and thus excluded from $M$. Specifically, during dynamic writes, the peak sizes of $\mathcal{B}$ are $wZ{+}4\eta$ for `DOItemAssign` and $3Z{+}2$ for `DOPMerge`. Furthermore, `DOCompact` needs a working buffer of size up to $\alpha_{\mathrm{BM}}(N, \varepsilon/2, \delta)$, which asymptotically upper-bounds $\eta$ since $N \geq m{+}1$. To accommodate both the persistent state and the execution of these DO algorithms, the enclave memory requires $M = \Omega(\max\{\alpha_{\mathrm{BM}}(N, \varepsilon/2, \delta), Z\})$.

For the estimation error, $\alpha_{\mathrm{BM}}(N, \varepsilon/2, \delta) = O((\log N/\varepsilon) \cdot \max(\sqrt{\log N \log(N/\delta)}, \log(N/\delta))) = O((\log N/\varepsilon)\cdot \log D)$. Substituting $\varepsilon$ derived earlier, we obtain $\alpha_{\mathrm{BM}}(N, \varepsilon/2, \delta) = O(\log N \cdot \log^2 D)$ for a constant $\gamma$. Similarly, for $\gamma = B^\rho$, we have $\alpha_{\mathrm{BM}}(N, \varepsilon/2, \delta) = O(\log N \cdot B^\rho \log^2 D/\log B)$. In summary, it simplifies to $O(Z \log N)$, necessitating $M = \Omega(Z \log N)$. Given $B = \Omega(Z)$, we safely strengthen this to a more practical system bound $M = \Omega(B \log N)$. Notably, the dynamic write phase remains unaffected by the requirements of `Flexway-Osort`, as it is used only for in-enclave buffer sorting.

However, to support bulk loading, the external-memory variant of `Flexway-Osort` mandates the standard tall cache assumption $M = \Omega(B^2)$, which dominates the memory bound for dynamic writes. Thus, our design offers a flexible memory requirement: it demands $M = \Omega(B^2)$ specifically for high-speed initialization, but efficiently scales with $M = \Omega(B \log N)$ for later dynamic writes.

## 7 Evaluation

We compare SapphMap with baselines across operations, covering external I/O at scale and real workloads. We also analyze its sensitivity to key design, system, and privacy parameters. We address two questions: (1) Does SapphMap achieve a good trade-off between costly-but-FO and efficient-but-insecure designs in practice? (2) How sensitive is this trade-off to key configurations?

### 7.1 Experimental Setup

**Environment.** We use an Ubuntu 20.04 server with dual Intel Xeon Gold 6354 CPUs (3GHz), 1TB RAM (128GB Intel SGX v2 EPC), and a 2TB SATA SSD. To emulate resource constraints, we limit RAM via Docker and enclave memory via SGX maximum heap size, inducing two-tier swapping. Notably, our design is enclave-agnostic.

**Implementation.** We implement SapphMap in C++20 with practical optimizations. Code and artifacts are available on GitHub [44]. First, we restrict $\mathcal{T}_0$ to a small in-enclave region (e.g., 32KB–2MB) rather than the full enclave memory $M$, reducing oblivious scan overhead during queries. Such a buffer can fit in modern enclave memory (tens of MB to hundreds of GB). We also keep the topmost SapphMap levels entirely in the enclave. Second, we allocate 75% of the privacy budget across DP boundaries to minimize padded segment sizes, thus accelerating per-level scans during queries. Third, we align segment sizes to block-size multiples, eliminating cross-block I/O amplification.

**Workloads.** We evaluate SapphMap using both synthetic and real-world KV workloads, where records consist of 8-byte keys and 24-byte payloads. For scalability, range query efficiency, and parameter analysis, we utilize YCSB [16] to generate fixed-size initial records, reporting average time across $2^{16}$ queries. To measure workload execution cost, we preload $2^{26}$ YCSB records and run two $2^{26}$-operation traces: YCSB-A (50% update, 50% point) and YCSB-Mix (20% insert, 20% update, 20% delete, 35% point, 5% range). In addition, we run two real-world production traces (1.35B MetaKV [3] and 1.67B IBM-COS [30]) from scratch to capture workload lifecycles; each is truncated to $2^{27}$ operations to keep the evaluation manageable.

**Baselines.** We select four dynamic data structures as baselines:

- *NO-LSMTree*. An enclave-based non-oblivious (NO) LSM tree implemented by us, aligned with our core configurations (e.g., tree parameters and hybrid internal storage), using fence pointers and per-block Bloom filters to minimize external I/O.
- *NO-HashMap*. An enclave-based non-oblivious (NO) hash map implemented by us, featuring an internal open-addressing buffer that batch-flushes to a bucketed external table, using per-bucket Bloom filters and page-level indexes to accelerate point queries.
- *ENIGMAP* [75]. An enclave-based FO map based on the oblivious AVL tree [80] with a locality-friendly ORAM layout. We use its official open-source implementation [76] as the baseline. It outperforms prior baselines such as Oblix [62] and Signal [59].
- *H2O2RAM* [86]. A TDX-based hierarchical O2RAM with tailored oblivious hash schemes, providing FO map support with improved data locality and parallelization. As it reportedly outperforms ENIGMAP in the original TDX evaluation, we port its official implementation [87] to our SGX enclave setting.

**Default settings.** Unless noted, SapphMap uses a 512KB in-enclave buffer $\mathcal{T}_0$, a fan-out $\gamma = 2$, and a block size $B = 32$KB. We set $\varepsilon_{\text{tot}} = 4.0$ and $\delta_{\text{tot}} = 1/(2 \times N^{1.2}) < 2^{-20}$ (for $N \geq 10^5$), which is consistent with established DP and DO settings using a constant $\varepsilon$ and $\delta \ll 1/N$ [65, 66]. This configuration thus falls within a practical privacy–utility regime, with tighter privacy parameters requiring more noise and padding. The domain size is fixed at 100× the total data volume. We primarily evaluate SapphMap with 32GB enclave memory and 128GB RAM (denoted as 32G/128G), but scale down to a 256M/1G configuration to measure swap performance on smaller datasets.

### 7.2 Comparative Performance

*7.2.1 Data Scalability.* Figure 6 shows the initialization and point-query time for different data structures. Initialization includes bulk loading and incremental insertion, with point query time measured only on incrementally built structures to reflect real-world scenarios. Also, markers with black borders indicate the onset of swapping to untrusted RAM and, at larger scales, to disk.

SapphMap exhibits markedly better performance than its FO counterparts, ENIGMAP and H2O2RAM. It outperforms ENIGMAP by over 400× for incremental insertions and by 13× ~ 264× for point queries, while surpassing H2O2RAM by over 29× and 6.1× ~ 76×, respectively, with the largest gains on smaller datasets where external I/O is limited. We omit bulk-loading

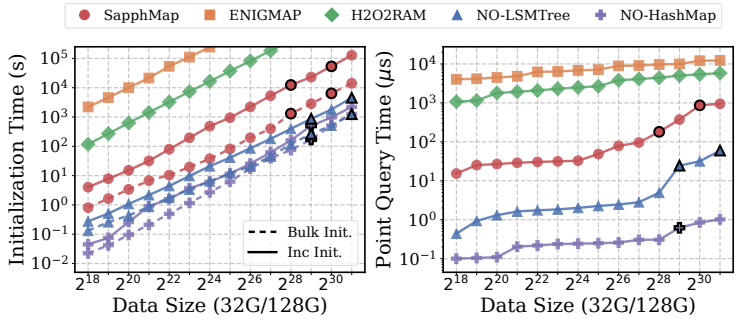



Fig. 6. Comparison of data structures for varying data sizes

evaluations for ENIGMAP and H2O2RAM because neither released codebase provides a bulk-loading implementation. Notably, even with its officially reported 8.7× bulk-loading speedup over incremental insertions, ENIGMAP's bulk-loading time would still far exceed that of SapphMap.

When data sizes surpass enclave memory, query times increase noticeably due to external I/O overhead from accessing untrusted storage. This transition occurs at $2^{28}$ for SapphMap, and $2^{29}$ for NO-LSMTree and NO-HashMap. The B+-trees introduced by SapphMap occupy at most 0.02% of its persistent storage. At very large scales (e.g., $2^{31}$), where all structures incur swapping, SapphMap's point-query gaps relative to NO-LSMTree and NO-HashMap stand at 15.9× and 932×, respectively. In contrast, ENIGMAP remains much farther from NO-LSMTree (206.4×) and NO-HashMap (12,105.3×), while H2O2RAM trails them by 97.6× and 5,725.5×. This proves DO remains efficient at scale, especially compared to costlier FO.

*7.2.2 Range Query Efficiency.* Figure 7 compares range query times of different data structures across varying query lengths. Lacking built-in support for range operations, ENIGMAP, H2O2RAM, and NO-HashMap must issue a separate point query for each element in the range, causing severe slowdowns. However, SapphMap natively supports range queries, achieving up to million-fold speedups over ENIGMAP and H2O2RAM, and thousand-fold over NO-HashMap.

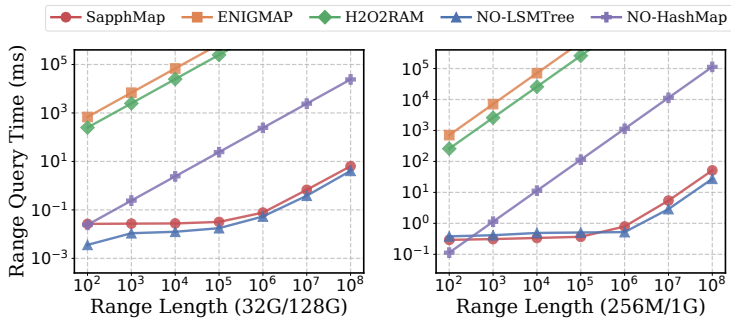



Fig. 7. Comparison across query lengths for data size = $2^{24}$

As query range grows, SapphMap's overhead over NO-LSMTree drops from 7.3× to 1.6× under 32G/128G. This occurs as the dominant cost shifts from segment lookups to item scanning; the residual 1.6× gap reflects SapphMap's inherent padding. Notably, while NO-LSMTree organizes items per block, SapphMap segments can span multiple blocks, lowering lookup costs and improving short-range query performance under 256M/1G. Unlike FO schemes, SapphMap achieves individual privacy without greatly sacrificing range query performance versus the insecure LSM-tree baseline.

*7.2.3 Workload Execution Cost.* Figure 8 depicts the execution time and space overhead across diverse workloads. SapphMap speeds up execution by 92.8×∼3226.2× over ENIGMAP and 16.1×∼1249.2× over H2O2RAM, with peak gains on the range-query workload. For space, it adds only 1.68× average overhead relative to NO-LSMTree and 1.3× to NO-HashMap. On read-dominant production traces MetaKV and IBM-COS, it uses as little as 0.42% of ENIGMAP's space and 34.6% of H2O2RAM's. The stark contrast to ENIGMAP occurs because ENIGMAP must provision for the worst-case entry count implied by the maximum number of operations when the true bound is unknown. Overall, SapphMap maintains high efficiency on diverse workloads, greatly narrowing the practicality gap between strict FO designs and performant but insecure data structures.

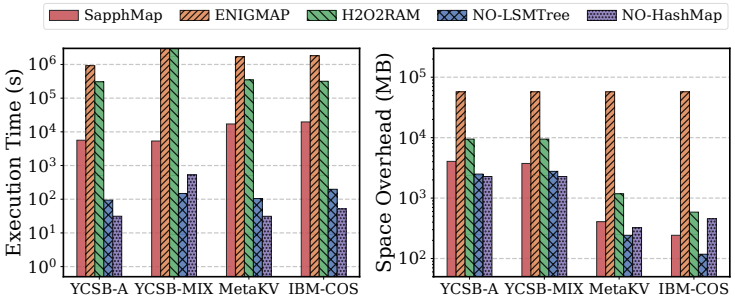



Fig. 8. Comparison of data structures across workloads

## 7.3 Parameter Sensitivity Analysis

Below, we study how key configurations affect SapphMap's amortized incremental insertion cost and average point-query time.

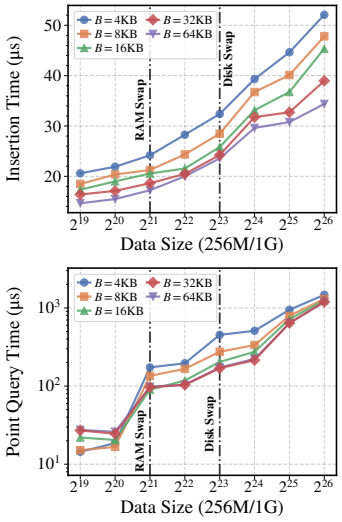



Fig. 9. Impact of block size $B$ for varying data sizes

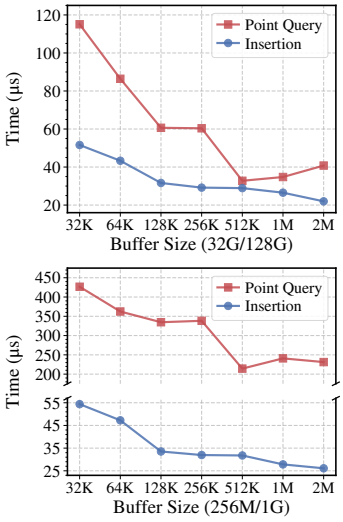



Fig. 10. Impact of $\mathcal{T}_0$ buffer size for data size = $2^{24}$

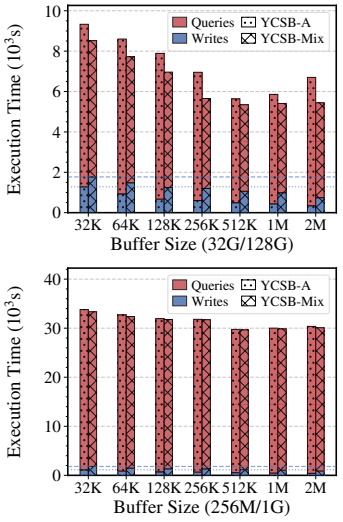



Fig. 11. Impact of $\mathcal{T}_0$ buffer size across mixed workloads

*7.3.1 Block Size.* Figure 9 shows how block size (including security metadata) affects performance. For insertions, larger blocks improve performance. In our implementation, intermediate results of

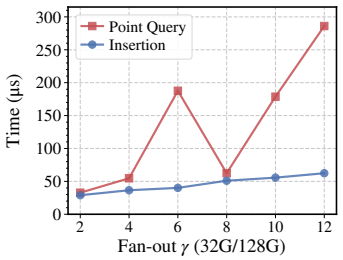
`DOItemAssign` and `DOPartition` are offloaded to untrusted memory to conserve enclave memory. Hence, larger blocks lower I/O frequency, thus mitigating costly enclave cryptographic overhead.

For point queries, smaller blocks are preferable before RAM swapping, as large-block alignment inflates segment sizes and increases the in-enclave oblivious scan time at each level. Once swapping occurs, however, larger blocks become advantageous by reducing the I/O frequency for segment fetches. 32KB strikes a good balance, limiting alignment overhead while minimizing I/O costs.

*7.3.2 Buffer Size.* Figure 10 depicts the buffer-size effects on performance at data size $2^{24}$. Increasing buffer size accelerates insertions by maximizing sequential writes and reducing `DOFlush` frequency, thereby mitigating both in-enclave processing and I/O overheads.

For point queries, buffer size presents a U-shaped trade-off: a larger buffer decreases SapphMap's height, allowing higher privacy budgets for DP boundaries and thus reducing segment sizes and hierarchical scan costs. Conversely, it raises the cost of oblivious scans within the in-enclave buffer, yielding a sweet spot at 512KB.

Figure 11 evaluates this insertion–query trade-off on the YCSB-A and YCSB-Mix workloads. Queries dominate execution time because they constitute a significant proportion of both workloads and are costlier than amortized batched writes. Hence, all scenarios exhibit a similar U-shaped trend, confirming that the default choice of a 512KB buffer strikes a good balance. This default suits typical workloads, but a larger buffer should be considered for highly write-intensive workloads. Oversized buffers incur costly oblivious buffer scans, while overly small ones cause frequent flushes and a deeper hierarchy. Even so, performance degrades gracefully across both memory settings as the buffer shrinks to 32KB.



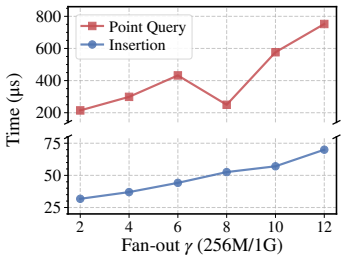
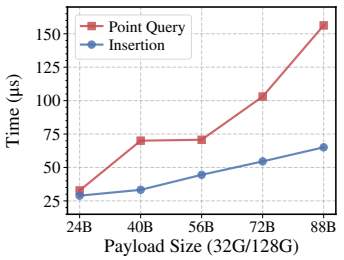




Fig. 12. Impact of fan-out $\gamma$ for data size = $2^{24}$



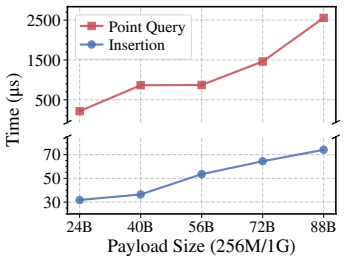
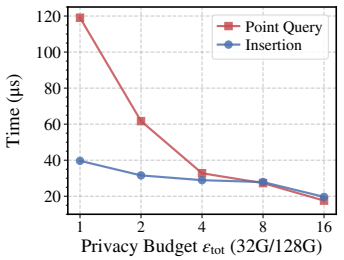




Fig. 13. Impact of payload size for data size = $2^{24}$



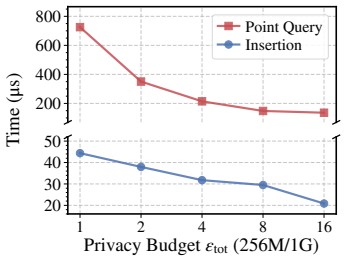




Fig. 14. Impact of privacy budget $\varepsilon_{\text{tot}}$ for data size = $2^{24}$

*7.3.3 Fan-out.* Figure 12 plots the performance sensitivity to the fan-out $\gamma$. For insertions, as in standard LSM-trees, a larger fan-out ($\gamma \geq 4$) raises amortized costs, since the heavier per-level `DOPMerge` overhead outweighs the benefits of fewer levels. It also increases the maximum number of PDS constructions ($\mu$) an item can undergo, shrinking the privacy budget allocated to each DO component. This, in turn, forces larger working buffers and adds compute overhead.

For point queries, complex factors determine how $\gamma$ affects them. While a larger $\gamma$ yields a shallower SapphMap, it lowers the probability ($1/\gamma$) of skipping a level. This partly offsets the depth

gains, as obliviousness requires accessing every level. Also, segment sizes increase due to smaller privacy budget for DP boundaries. Thus, a small fan-out ($\gamma = 2$) suffices for optimal performance.

*7.3.4 Payload Size.* Figure 13 evaluates the impact of record payload size, showing that larger payloads generally degrade insertion and point-query efficiency. This occurs because larger payloads inflate record and segment sizes, thereby raising PDS construction and scan costs. Furthermore, they reduce the item capacity of the fixed-size in-enclave buffer, triggering more frequent `DOFlush` operations and further increasing the amortized insertion cost.

*7.3.5 Privacy Budget.* Figure 14 presents the effect of the total privacy budget $\varepsilon_{\text{tot}}$ on performance, showing that a higher $\varepsilon_{\text{tot}}$ consistently accelerates both insertions and point queries. This is because a larger budget relaxes the noise requirements for the data structure, leading to smaller segment sizes and reduced working buffer capacities for obfuscation, which lower computational overhead.

## 8 Additional Related Work

*ORAM and Oblivious Algorithms.* Oblivious RAM (ORAM) was first formalized by Goldreich [33, 34] to provide FO memory access patterns. Since then, a range of improved schemes have been proposed, such as Path ORAM [73], Circuit ORAM [79], and Onion ORAM [18]. In parallel, to bypass the polylog per-operation overhead of generic ORAM, custom FO algorithms have been developed with better asymptotic bounds or running efficiency. Notable examples include oblivious sorting [2, 36, 37, 41], compaction [20, 35, 70], random permutation [2, 41, 46, 70], selection [35, 57], and database joins [12, 47, 54, 61, 81]. In recent years, DO algorithms [82, 90, 91] have also been explored to further improve efficiency.

*Oblivious Data Structures.* Unlike one-off oblivious algorithms, oblivious data structures are inherently stateful. Prior work has developed various FO data structures, such as stacks [80], queues [80], priority queues [50, 72], and inverted file R-trees [85]. In particular, oblivious maps serve as indexing structures for efficient queries, avoiding the costly scan-based selection commonly used for exact-match data retrieval in FO [29, 61, 88] and DO [65] databases.

Separately, a distinct line of oblivious map designs [7, 42, 69, 77, 80] targets the traditional client-server setting. In this model, the ORAM client is assumed to be fully trusted, and its internal memory access patterns are not exposed to the adversary. By contrast, our enclave setting requires double obliviousness: since the client logic resides inside the enclave on an untrusted server, the client-side algorithm must also be oblivious. Consequently, adapting these designs to our setting would require non-trivial algorithmic changes, making direct adoption or comparison less meaningful.

## 9 Conclusion

In this paper, we presented SapphMap, the first highly efficient, dynamic, external-memory DO map supporting range queries for enclaves. By combining an LSM-inspired hierarchy with DO building blocks based on Private Partition, SapphMap matches the asymptotic I/O complexity of standard non-oblivious leveled LSM-trees for both writes and queries. Our evaluation shows SapphMap achieves a practical balance between the strong privacy guarantees of FO designs and the efficiency of non-oblivious ones, making it a promising solution for privacy-aware real-world data applications. Future work includes parallelizing SapphMap, extending it to distributed settings, and integrating it with production storage systems.

## Acknowledgments

This work is supported in part by HKRGC under grants 16204223 and 16203924.

## References


[1] Tristan Allard, Louis Béziaud, and Sébastien Gambs. 2020. Publication of Court Records: Circumventing the Privacy-Transparency Trade-Off. In *AICOL (Lecture Notes in Computer Science, Vol. 13048)*. Springer, 298–312.

[2] Gilad Asharov, T.-H. Hubert Chan, Kartik Nayak, Rafael Pass, Ling Ren, and Elaine Shi. 2020. Bucket Oblivious Sort: An Extremely Simple Oblivious Sort. In *SOSA*. SIAM, 8–14.

[3] Benjamin Berg, Daniel S. Berger, Sara McAllister, Isaac Grosof, Sathya Gunasekar, Jimmy Lu, Michael Uhlar, Jim Carrig, Nathan Beckmann, Mor Harchol-Balter, and Gregory R. Ganger. 2020. The CacheLib Caching Engine: Design and Experiences at Scale. In *OSDI*. USENIX Association, 753–768.

[4] Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. 2013. *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press. doi:10.1093/acprof:oso/9780199535255.001.0001

[5] Ferdinand Brasser, Urs Müller, Alexandra Dmitrienko, Kari Kostiainen, Srdjan Capkun, and Ahmad-Reza Sadeghi. 2017. Software Grand Exposure: SGX Cache Attacks Are Practical. In *WOOT*. USENIX Association.

[6] Mark Bun, Kobbi Nissim, Uri Stemmer, and Salil P. Vadhan. 2015. Differentially Private Release and Learning of Threshold Functions. In *FOCS*. IEEE Computer Society, 634–649.

[7] Xinle Cao, Weiqi Feng, Jian Liu, Jinjin Zhou, Wenjing Fang, Lei Wang, Quanqing Xu, Chuanhui Yang, and Kui Ren. 2024. Towards Practical Oblivious Map. *Proc. VLDB Endow.* 18, 3 (2024), 688–701.

[8] Matteo Ceccarello, Anton Dignös, Johann Gamper, and Christina Khnaisser. 2023. Indexing Temporal Relations for Range-Duration Queries. In *SSDBM*. ACM, 3:1–3:12.

[9] T.-H. Hubert Chan, Kai-Min Chung, Bruce M. Maggs, and Elaine Shi. 2019. Foundations of Differentially Oblivious Algorithms. In *SODA*. SIAM, 2448–2467.

[10] T.-H. Hubert Chan, Yue Guo, Wei-Kai Lin, and Elaine Shi. 2018. Cache-Oblivious and Data-Oblivious Sorting and Applications. In *SODA*. SIAM, 2201–2220.

[11] T.-H. Hubert Chan, Elaine Shi, and Dawn Song. 2010. Private and Continual Release of Statistics. In *ICALP (2) (Lecture Notes in Computer Science, Vol. 6199)*. Springer, 405–417.

[12] Zhao Chang, Dong Xie, Sheng Wang, and Feifei Li. 2022. Towards Practical Oblivious Join. In *SIGMOD Conference*. ACM, 803–817.

[13] Bernard Chazelle and Leonidas J. Guibas. 1986. Fractional Cascading: I. A Data Structuring Technique. *Algorithmica* 1, 2 (1986), 133–162.

[14] Bernard Chazelle and Leonidas J. Guibas. 1986. Fractional Cascading: II. Applications. *Algorithmica* 1, 2 (1986), 163–191.

[15] Sanchuan Chen, Xiaokuan Zhang, Michael K. Reiter, and Yinqian Zhang. 2017. Detecting Privileged Side-Channel Attacks in Shielded Execution with Déjà Vu. In *AsiaCCS*. ACM, 7–18.

[16] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *SoCC*. ACM, 143–154.

[17] Victor Costan and Srinivas Devadas. 2016. Intel SGX Explained. *IACR Cryptol. ePrint Arch.* 2016 (2016), 86.

[18] Srinivas Devadas, Marten van Dijk, Christopher W. Fletcher, Ling Ren, Elaine Shi, and Daniel Wichs. 2016. Onion ORAM: A Constant Bandwidth Blowup Oblivious RAM. In *TCC (A2) (Lecture Notes in Computer Science)*. Springer, 145–174.

[19] Jean-François Dhem, François Koeune, Philippe-Alexandre Leroux, Patrick Mestré, Jean-Jacques Quisquater, and Jean-Louis Willems. 1998. A Practical Implementation of the Timing Attack. In *CARDIS (Lecture Notes in Computer Science, Vol. 1820)*. Springer, 167–182.

[20] Samuel Dittmer and Rafail Ostrovsky. 2020. Oblivious Tight Compaction In O(n) Time with Smaller Constant. In *SCN (Lecture Notes in Computer Science)*. Springer, 253–274.

[21] Shi Dong, Khushnood Abbas, and Raj Jain. 2019. A Survey on Distributed Denial of Service (DDoS) Attacks in SDN and Cloud Computing Environments. *IEEE Access* 7 (2019), 80813–80828.

[22] Siying Dong, Andrew Kryczka, Yanqin Jin, and Michael Stumm. 2021. RocksDB: Evolution of Development Priorities in a Key-value Store Serving Large-scale Applications. *ACM Trans. Storage* 17, 4 (2021), 26:1–26:32.

[23] Cynthia Dwork. 2006. Differential Privacy. In *ICALP (2) (Lecture Notes in Computer Science, Vol. 4052)*. Springer, 1–12.

[24] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam D. Smith. 2006. Calibrating Noise to Sensitivity in Private Data Analysis. In *TCC (Lecture Notes in Computer Science, Vol. 3876)*. Springer, 265–284.

[25] Cynthia Dwork, Moni Naor, Omer Reingold, and Guy N. Rothblum. 2015. Pure Differential Privacy for Rectangle Queries via Private Partitions. In *ASIACRYPT (2) (Lecture Notes in Computer Science, Vol. 9453)*. Springer, 735–751.

[26] Cynthia Dwork and Aaron Roth. 2014. The Algorithmic Foundations of Differential Privacy. *Found. Trends Theor. Comput. Sci.* 9, 3-4 (2014), 211–407.

[27] Cynthia Dwork, Guy N. Rothblum, and Salil P. Vadhan. 2010. Boosting and Differential Privacy. In *FOCS*. IEEE Computer Society, 51–60.

[28] Muhammad El-Hindi, Tobias Ziegler, Matthias Heinrich, Adrian Lutsch, Zheguang Zhao, and Carsten Binnig. 2022. Benchmarking the Second Generation of Intel SGX Hardware. In *DaMoN*. ACM, 5:1–5:8.

[29] Saba Eskandarian and Matei Zaharia. 2019. ObliDB: Oblivious Query Processing for Secure Databases. *Proc. VLDB Endow.* 13, 2 (2019), 169–183.

[30] Ohad Eytan, Danny Harnik, Effi Ofer, Roy Friedman, and Ronen I. Kat. 2020. It's Time to Revisit LRU vs. FIFO. In *HotStorage*. USENIX Association.

[31] Christopher W. Fletcher, Ling Ren, Xiangyao Yu, Marten van Dijk, Omer Khan, and Srinivas Devadas. 2014. Suppressing the Oblivious RAM timing channel while making information leakage and program efficiency trade-offs. In *HPCA*. IEEE Computer Society, 213–224.

[32] Simson L. Garfinkel, John M. Abowd, and Sarah Powazek. 2018. Issues Encountered Deploying Differential Privacy. In *WPES@CCS*. ACM, 133–137.

[33] Oded Goldreich. 1987. Towards a Theory of Software Protection and Simulation by Oblivious RAMs. In *STOC*. ACM, 182–194.

[34] Oded Goldreich and Rafail Ostrovsky. 1996. Software Protection and Simulation on Oblivious RAMs. *J. ACM* 43, 3 (1996), 431–473.

[35] Michael T. Goodrich. 2011. Data-oblivious external-memory algorithms for the compaction, selection, and sorting of outsourced data. In *SPAA*. ACM, 379–388.

[36] Michael T. Goodrich. 2011. Randomized Shellsort: A Simple Data-Oblivious Sorting Algorithm. *J. ACM* 58, 6 (2011), 27:1–27:26.

[37] Michael T. Goodrich. 2014. Zig-zag sort: a simple deterministic data-oblivious sorting algorithm running in O(n log n) time. In *STOC*. ACM, 684–693.

[38] Johannes Götzfried, Moritz Eckert, Sebastian Schinzel, and Tilo Müller. 2017. Cache Attacks on Intel SGX. In *EUROSEC*. ACM, 2:1–2:6.

[39] Paul Grubbs, Marie-Sarah Lacharité, Brice Minaud, and Kenneth G. Paterson. 2019. Learning to Reconstruct: Statistical Learning Theory and Encrypted Database Attacks. In *IEEE Symposium on Security and Privacy*. IEEE, 1067–1083.

[40] Daniel Gruss, Julian Lettner, Felix Schuster, Olga Ohrimenko, István Haller, and Manuel Costa. 2017. Strong and Efficient Cache Side-Channel Protection using Hardware Transactional Memory. In *USENIX Security Symposium*. USENIX Association, 217–233.

[41] Tianyao Gu, Yilei Wang, Afonso Tinoco, Bingnan Chen, Ke Yi, and Elaine Shi. 2025. Flexway O-Sort: Enclave-Friendly and Optimal Oblivious Sorting. In *USENIX Security Symposium*. USENIX Association, 7563–7582.

[42] Yitong Guo, Hongbo Chen, Haobin Hiroki Chen, Yukui Luo, XiaoFeng Wang, and Chenghong Wang. 2025. BOLT: Bandwidth-Optimized Lightning-Fast Oblivious Map powered by Secure HBM Accelerators. In *CCS*. ACM, 2594–2608.

[43] Hakan Hacigümüs, Sharad Mehrotra, and Balakrishna R. Iyer. 2002. Providing Database as a Service. In *ICDE*. IEEE Computer Society, 29–38.

[44] Xiaoyu Han, Ke Yi, Wei Dong, and Xinying Yang. 2026. SapphMap: Code and Full Version. https://github.com/HatsuneHan/SapphMap-SIGMOD27.

[45] Michael Hay, Vibhor Rastogi, Gerome Miklau, and Dan Suciu. 2010. Boosting the Accuracy of Differentially Private Histograms Through Consistency. *Proc. VLDB Endow.* 3, 1 (2010), 1021–1032.

[46] William L. Holland, Olga Ohrimenko, and Anthony Wirth. 2022. Efficient Oblivious Permutation via the Waksman Network. In *AsiaCCS*. ACM, 771–783.

[47] Xiao Hu and Zhiang Wu. 2025. Optimal Oblivious Algorithms for Multi-Way Joins. In *ICDT (LIPIcs)*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 25:1–25:19.

[48] Dongxu Huang, Qi Liu, Qiu Cui, Zhuhe Fang, Xiaoyu Ma, Fei Xu, Li Shen, Liu Tang, Yuxing Zhou, Menglong Huang, Wan Wei, Cong Liu, Jian Zhang, Jianjun Li, Xuelian Wu, Lingyu Song, Ruoxi Sun, Shuaipeng Yu, Lei Zhao, Nicholas Cameron, Liquan Pei, and Xin Tang. 2020. TiDB: A Raft-based HTAP Database. *Proc. VLDB Endow.* 13, 12 (2020), 3072–3084.

[49] Mohammad Saiful Islam, Mehmet Kuzu, and Murat Kantarcioglu. 2012. Access Pattern disclosure on Searchable Encryption: Ramification, Attack and Mitigation. In *NDSS*. The Internet Society.

[50] Zahra Jafargholi, Kasper Green Larsen, and Mark Simkin. 2021. Optimal Oblivious Priority Queues. In *SODA*. SIAM, 2366–2383.

[51] Yeongjin Jang, Jae-Hyuk Lee, Sangho Lee, and Taesoo Kim. 2017. SGX-Bomb: Locking Down the Processor via Rowhammer Attack. In *SysTEX@SOSP*. ACM, 5:1–5:6.

[52] Georgios Kellaris, George Kollios, Kobbi Nissim, and Adam O'Neill. 2016. Generic Attacks on Secure Outsourced Databases. In *CCS*. ACM, 1329–1340.

[53] Deokjin Kim, DaeHee Jang, Minjoon Park, Yunjong Jeong, Jonghwan Kim, Seokjin Choi, and Brent ByungHoon Kang. 2019. SGX-LEGO: Fine-grained SGX controlled-channel attack and its countermeasure. *Comput. Secur.* 82 (2019), 118–139.

[54] Simeon Krastnikov, Florian Kerschbaum, and Douglas Stebila. 2020. Efficient Oblivious Database Joins. *Proc. VLDB Endow.* 13, 11 (2020), 2132–2145.
[55] Dayeol Lee, David Kohlbrenner, Shweta Shinde, Krste Asanovic, and Dawn Song. 2020. Keystone: an open framework for architecting trusted execution environments. In *EuroSys*. ACM, 38:1–38:16.
[56] Sangho Lee, Ming-Wei Shih, Prasun Gera, Taesoo Kim, Hyesoon Kim, and Marcus Peinado. 2017. Inferring Fine-grained Control Flow Inside SGX Enclaves with Branch Shadowing. In *USENIX Security Symposium*. USENIX Association, 557–574.
[57] Frank Thomson Leighton, Yuan Ma, and Torsten Suel. 1995. On Probabilistic Networks for Selection, Merging, and Sorting. In *SPAA*. ACM, 106–118.
[58] Min Lyu, Dong Su, and Ninghui Li. 2017. Understanding the Sparse Vector Technique for Differential Privacy. *Proc. VLDB Endow.* 10, 6 (2017), 637–648.
[59] Moxie Marlinspike. 2017. Technology preview: Private contact discovery for Signal. https://signal.org/blog/private-contact-discovery/.
[60] Peter Mattis. 2020. Introducing Pebble: A RocksDB-Inspired Key-Value Store Written in Go. Cockroach Labs Blog. https://www.cockroachlabs.com/blog/pebble-rocksdb-kv-store/
[61] Apostolos Mavrogiannakis, Xian Wang, Ioannis Demertzis, Dimitrios Papadopoulos, and Minos N. Garofalakis. 2025. OBLIVIATOR: OBLIVIous Parallel Joins and other OperATORs in Shared Memory Environments. In *USENIX Security Symposium*. USENIX Association, 8521–8540.
[62] Pratyush Mishra, Rishabh Poddar, Jerry Chen, Alessandro Chiesa, and Raluca Ada Popa. 2018. Oblix: An Efficient Oblivious Search Index. In *IEEE Symposium on Security and Privacy*. IEEE Computer Society, 279–296.
[63] Ahmad Moghimi, Gorka Irazoqui, and Thomas Eisenbarth. 2017. CacheZoom: How SGX Amplifies the Power of Cache Attacks. In *CHES (Lecture Notes in Computer Science, Vol. 10529)*. Springer, 69–90.
[64] Omiros Papaspiliopoulos. 2020. High-dimensional probability: An introduction with applications in data science.
[65] Lianke Qin, Rajesh Jayaram, Elaine Shi, Zhao Song, Danyang Zhuo, and Shumo Chu. 2022. Adore: Differentially Oblivious Relational Database Operators. *CoRR* abs/2212.05176 (2022).
[66] Lina Qiu, Georgios Kellaris, Nikos Mamoulis, Kobbi Nissim, and George Kollios. 2023. Doquet: Differentially Oblivious Range and Join Queries with Private Data Structures. *Proc. VLDB Endow.* 16, 13 (2023), 4160–4173.
[67] Yuan Qiu and Ke Yi. 2022. Differential Privacy on Dynamic Data. *CoRR* abs/2209.01387 (2022).
[68] Vijaya Ramachandran and Elaine Shi. 2021. Data Oblivious Algorithms for Multicores. In *SPAA*. ACM, 373–384.
[69] Daniel S. Roche, Adam J. Aviv, and Seung Geol Choi. 2016. A Practical Oblivious Map Data Structure with Secure Deletion and History Independence. In *IEEE Symposium on Security and Privacy*. IEEE Computer Society, 178–197.
[70] Sajin Sasy, Aaron Johnson, and Ian Goldberg. 2022. Fast Fully Oblivious Compaction and Shuffling. In *CCS*. ACM, 2565–2579.
[71] Michael Schwarz, Samuel Weiser, Daniel Gruss, Clémentine Maurice, and Stefan Mangard. 2017. Malware Guard Extension: Using SGX to Conceal Cache Attacks. In *DIMVA (Lecture Notes in Computer Science, Vol. 10327)*. Springer, 3–24.
[72] Elaine Shi. 2020. Path Oblivious Heap: Optimal and Practical Oblivious Priority Queue. In *SP*. IEEE, 842–858.
[73] Emil Stefanov, Marten van Dijk, Elaine Shi, Christopher W. Fletcher, Ling Ren, Xiangyao Yu, and Srinivas Devadas. 2013. Path ORAM: an extremely simple oblivious RAM protocol. In *CCS*. ACM, 299–310.
[74] Rebecca Taft, Irfan Sharif, Andrei Matei, Nathan VanBenschoten, Jordan Lewis, Tobias Grieger, Kai Niemi, Andy Woods, Anne Birzin, Raphael Poss, Paul Bardea, Amruta Ranade, Ben Darnell, Bram Gruneir, Justin Jaffray, Lucy Zhang, and Peter Mattis. 2020. CockroachDB: The Resilient Geo-Distributed SQL Database. In *SIGMOD Conference*. ACM, 1493–1509.
[75] Afonso Tinoco, Sixiang Gao, and Elaine Shi. 2023. EnigMap: External-Memory Oblivious Map for Secure Enclaves. In *USENIX Security Symposium*. USENIX Association, 4033–4050.
[76] Afonso Tinoco, Sixiang Gao, and Elaine Shi. 2023. ENIGMAP Source Code. https://github.com/odslib/EnigMap.
[77] Ruixuan Wang, Siyi Lv, Xiang Li, Haoshuai Gong, Zheli Liu, Tong Li, and Liang Guo. 2026. BOMAP: A Round-Efficient Construction of Oblivious Maps. *IEEE Trans. Dependable Secur. Comput.* 23, 1 (2026), 221–235.
[78] Wenhao Wang, Guoxing Chen, Xiaorui Pan, Yinqian Zhang, XiaoFeng Wang, Vincent Bindschaedler, Haixu Tang, and Carl A. Gunter. 2017. Leaky Cauldron on the Dark Land: Understanding Memory Side-Channel Hazards in SGX. In *CCS*. ACM, 2421–2434.
[79] Xiao Wang, T.-H. Hubert Chan, and Elaine Shi. 2015. Circuit ORAM: On Tightness of the Goldreich-Ostrovsky Lower Bound. In *CCS*. ACM, 850–861.
[80] Xiao Shaun Wang, Kartik Nayak, Chang Liu, T.-H. Hubert Chan, Elaine Shi, Emil Stefanov, and Yan Huang. 2014. Oblivious Data Structures. In *CCS*. ACM, 215–226.
[81] Yilei Wang, Xiangdong Zeng, Sheng Wang, and Feifei Li. 2025. Jodes: Efficient Oblivious Join in the Distributed Setting. *Proc. VLDB Endow.* 18, 5 (2025), 1291–1304.

[82] Zhiang Wu, Wei Dong, and Xiao Hu. 2026. Differentially Oblivious Multi-way Join. *Proc. ACM Manag. Data* 4, 2 (2026), 162.
[83] Yuanzhong Xu, Weidong Cui, and Marcus Peinado. 2015. Controlled-Channel Attacks: Deterministic Side Channels for Untrusted Operating Systems. In *IEEE Symposium on Security and Privacy*. IEEE Computer Society, 640–656.
[84] Qiao Yan, F. Richard Yu, Qingxiang Gong, and Jianqiang Li. 2016. Software-Defined Networking (SDN) and Distributed Denial of Service (DDoS) Attacks in Cloud Computing Environments: A Survey, Some Research Issues, and Challenges. *IEEE Commun. Surv. Tutorials* 18, 1 (2016), 602–622.
[85] Zikai Ye, Xiangyu Wang, Zesen Liu, Dan Zhu, and Jianfeng Ma. 2025. OBIR-tree: An Efficient Oblivious Index for Spatial Keyword Queries on Secure Enclaves. *Proc. ACM Manag. Data* 3, 1 (2025), 58:1–58:24.
[86] Leqian Zheng, Zheng Zhang, Wentao Dong, Yao Zhang, Ye Wu, and Cong Wang. 2025. H2O2RAM: A High-Performance Hierarchical Doubly Oblivious RAM. In *USENIX Security Symposium*. USENIX Association, 8501–8520.
[87] Leqian Zheng, Zheng Zhang, Wentao Dong, Yao Zhang, Ye Wu, and Cong Wang. 2025. H2O2RAM Source Code. https://doi.org/10.5281/zenodo.14648338.
[88] Wenting Zheng, Ankur Dave, Jethro G. Beekman, Raluca Ada Popa, Joseph E. Gonzalez, and Ion Stoica. 2017. Opaque: An Oblivious and Encrypted Distributed Analytics Platform. In *NSDI*. USENIX Association, 283–298.
[89] Ence Zhou, Haoli Sun, Bingfeng Pi, Jun Sun, Kazuhiro Yamashita, and Yoshihide Nomura. 2019. Ledgerdata Refiner: A Powerful Ledger Data Query Platform for Hyperledger Fabric. In *IoTSMS*. IEEE, 433–440.
[90] Mingxun Zhou, Elaine Shi, T.-H. Hubert Chan, and Shir Maimon. 2023. A Theory of Composition for Differential Obliviousness. In *EUROCRYPT (3) (Lecture Notes in Computer Science, Vol. 14006)*. Springer, 3–34.
[91] Mingxun Zhou, Mengshi Zhao, T.-H. Hubert Chan, and Elaine Shi. 2024. Advanced Composition Theorems for Differential Obliviousness. In *ITCS (LIPIcs)*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 103:1–103:24.

# A Deferred Proofs

## A.1 Proof of Lemma 3.7

To prove Lemma 3.7, we first establish the following Bernstein inequality for sums of independent Laplace random variables.

Lemma A.1 (Bernstein Inequality for Laplace Sums). *Let $Y_1, \ldots, Y_k$ be independent random variables drawn from* $\mathrm{Lap}(\lambda)$. *Let $\|Y_i\|_{\psi_1} = 2\lambda$ denote the sub-exponential norm of a* $\mathrm{Lap}(\lambda)$ *random variable. For any $\zeta > 0$, the following Bernstein inequality holds:*

$$\Pr\left[\left|\sum_{i=1}^{k} Y_i\right| > \zeta\right] \leq 2\exp\left(-\frac{1}{2}\min\left\{\frac{\zeta^2}{k\|Y_i\|_{\psi_1}^2}, \frac{\zeta}{\|Y_i\|_{\psi_1}}\right\}\right).$$

Proof. The proof is based on establishing the parameters for the Bernstein inequality as stated in Theorem 2.10 of the book [4].

First, we determine the variance parameter $v$ and the scale parameter $c$ required for the Bernstein inequality [4, 64]. The variance condition necessitates that $\sum_{i=1}^{k} \mathbb{E}[Y_i^2] \leq v$. For $Y_i \sim \mathrm{Lap}(\lambda)$, the second moment is given by $\mathbb{E}[Y_i^2] = 2\lambda^2$. Summing over the $k$ independent variables, we have $\sum_{i=1}^{k} \mathbb{E}[Y_i^2] = 2k\lambda^2$. Consequently, we can readily set $v = 2k\lambda^2$ to satisfy the variance condition.

For the scale parameter $c$, it must satisfy the Bernstein moment condition for all integers $q \geq 3$: $\sum_{i=1}^{k} \mathbb{E}[|Y_i|^q] \leq \frac{q!}{2} v c^{q-2}$. Recall that for Laplace distribution, $\mathbb{E}[|Y_i|^q] = q!\lambda^q$. The sum of moments is:

$$\sum_{i=1}^{k} \mathbb{E}[|Y_i|^q] = kq!\lambda^q = q!k\lambda^2\lambda^{q-2}.$$

Substituting $v = 2k\lambda^2$ (which implies $k\lambda^2 = v/2$), we obtain:

$$\sum_{i=1}^{k} \mathbb{E}[|Y_i|^q] = \frac{q!}{2} v\lambda^{q-2}.$$

Comparing this to the moment condition, we require $\lambda^{q-2} \leq c^{q-2}$. Hence, we select the scale parameter $c = \lambda$.

Second, we derive a concentration bound for the sum $S = \sum_{i=1}^{k} Y_i$ in terms of the parameters $v$ and $c$. For variables satisfying the Bernstein conditions $(v, c)$, and for all $t \in (0, 1/c)$, the logarithmic moment generating function (MGF) $\psi_S(t)$ is bounded by:

$$\psi_S(t) \leq \frac{vt^2}{2(1-ct)}.$$

Exponentiating this inequality yields the bound for the MGF $\mathbb{E}[e^{tS}]$:

$$\mathbb{E}[e^{tS}] \leq \exp\left(\frac{vt^2}{2(1-ct)}\right).$$

By applying Markov's inequality to $e^{tS}$, we obtain the Chernoff bound for any $\zeta > 0$ and $t \in (0, 1/c)$:

$$\Pr[S > \zeta] \leq e^{-t\zeta}\mathbb{E}[e^{tS}] \leq \exp\left(-t\zeta + \frac{vt^2}{2(1-ct)}\right).$$

We choose $t = \zeta/(v + c\zeta)$, which satisfies $0 < t < 1/c$. Substituting this value simplifies the above exponent:

$$-t\zeta + \frac{vt^2}{2(1-ct)} = -\frac{\zeta^2}{v+c\zeta} + \frac{\zeta^2}{2(v+c\zeta)} = -\frac{\zeta^2}{2(v+c\zeta)}.$$

By symmetry of the Laplace distribution, the two-sided bound is:

$$\Pr[|S| > \zeta] \leq 2\exp\left(-\frac{\zeta^2}{2(v + c\zeta)}\right).$$

Substituting $v = 2k\lambda^2$ and $c = \lambda$, and using the notation $\|Y_i\|_{\psi_1} = 2\lambda$:

$$\Pr[|S| > \zeta] \leq 2\exp\left(-\frac{\zeta^2}{k\|Y_i\|_{\psi_1}^2 + \zeta\|Y_i\|_{\psi_1}}\right).$$

Finally, to derive the simplified sub-exponential bound, we employ the inequality $\frac{1}{A+B} \geq \frac{1}{2}\min\left(\frac{1}{A}, \frac{1}{B}\right)$, which holds for any $A, B > 0$. Setting $A = k\|Y_i\|_{\psi_1}^2/\zeta^2$ and $B = \|Y_i\|_{\psi_1}/\zeta$, we obtain:

$$-\frac{\zeta^2}{k\|Y_i\|_{\psi_1}^2 + \zeta\|Y_i\|_{\psi_1}} \leq -\frac{1}{2}\min\left\{\frac{\zeta^2}{k\|Y_i\|_{\psi_1}^2}, \frac{\zeta}{\|Y_i\|_{\psi_1}}\right\}.$$

Substituting this exponent bound back into the probability estimate for $S = \sum_{i=1}^{k} Y_i$ completes the proof of Lemma A.1:

$$\Pr\left[\left|\sum_{i=1}^{k} Y_i\right| > \zeta\right] \leq 2\exp\left(-\frac{1}{2}\min\left\{\frac{\zeta^2}{k\|Y_i\|_{\psi_1}^2}, \frac{\zeta}{\|Y_i\|_{\psi_1}}\right\}\right).$$

□

With the general Bernstein inequality established in Lemma A.1, we are now ready to prove the main concentration bound:

Lemma 3.7 (Concentration Bound for Laplace Sums). *Let $Y_1, \ldots, Y_k$ be independent random variables drawn from* $\mathrm{Lap}(\lambda)$. *For any failure probability $\beta \in (0, 1)$, the following tail bound holds:*

$$\Pr\left[\left|\textstyle\sum_{i=1}^{k} Y_i\right| > \alpha_{\mathrm{Lap}}^{(k)}(\lambda, \beta)\right] \leq \beta,$$
$$\text{where } \alpha_{\mathrm{Lap}}^{(k)}(\lambda, \beta) = \lambda \cdot \max\left\{2\sqrt{2k\ln(2/\beta)}, 4\ln(2/\beta)\right\}.$$

Proof. We seek to find a threshold $\zeta$ such that the tail probability of the sum is at most $\beta$. Invoking the Bernstein Inequality (Lemma A.1) with norm $\|Y_i\|_{\psi_1} = 2\lambda$, it suffices to require:

$$2\exp\left(-\frac{1}{2}\min\left\{\frac{\zeta^2}{4k\lambda^2}, \frac{\zeta}{2\lambda}\right\}\right) \leq \beta.$$

Rearranging the terms, this condition is equivalent to:

$$-\frac{1}{2}\min\left\{\frac{\zeta^2}{4k\lambda^2}, \frac{\zeta}{2\lambda}\right\} \leq \ln\left(\frac{\beta}{2}\right) \iff \min\left\{\frac{\zeta^2}{4k\lambda^2}, \frac{\zeta}{2\lambda}\right\} \geq 2\ln\left(\frac{2}{\beta}\right).$$

To ensure the minimum of these two terms is at least $2\ln(2/\beta)$, we require both terms to individually meet or exceed this value. Hence, we derive the lower bounds for $\zeta$ from each term separately.

First, considering the sub-Gaussian term, we need:

$$\frac{\zeta^2}{4k\lambda^2} \geq 2\ln\left(\frac{2}{\beta}\right) \implies \zeta \geq \lambda \cdot 2\sqrt{2k\ln(2/\beta)}.$$

Second, considering the sub-exponential term, we require:

$$\frac{\zeta}{2\lambda} \geq 2\ln\left(\frac{2}{\beta}\right) \implies \zeta \geq \lambda \cdot 4\ln(2/\beta).$$

To satisfy both conditions simultaneously, $\zeta$ must be at least the maximum of these two derived bounds. Hence, we choose:

$$\zeta = \lambda \cdot \max\left\{2\sqrt{2k\ln(2/\beta)}, 4\ln(2/\beta)\right\} = \alpha_{\text{Lap}}^{(k)}(\lambda, \beta).$$

This completes the proof. □

### A.2 Proof of Lemma 3.8

Lemma 3.8 (Estimation Error of Binary Mechanism). *Let $\mathcal{F}$ (with entries $f_i$) be the true prefix sums of a length-$T$ data stream, and $\tilde{\mathcal{F}}$ be the noisy ones by $\varepsilon$-DP BM. Then w.p. at least $1-\beta$, the estimation error satisfies $|\tilde{f}_i - f_i| \le \alpha_{\text{BM}}(T, \varepsilon, \beta)$ uniformly for all $i = 1, \ldots, T$, where*

$$\alpha_{\text{BM}}(T, \varepsilon, \beta) = \alpha_{\text{Lap}}^{(\lceil \log_2 T \rceil)}\left((\lfloor \log_2 T \rfloor + 1)/\varepsilon, \beta/T\right).$$

Proof. To prove Lemma 3.8, we analyze the noise distribution introduced by the Binary Mechanism and apply the concentration bound for Laplace sums established in Lemma 3.7.

First, we determine the noise scale parameter $\lambda$ required to satisfy $\varepsilon$-DP. The Binary Mechanism maintains partial sums in a binary interval tree, where each node represents the sum of a specific interval. For a data stream of length $T$, changing a single entry by at most 1 affects only the nodes along the path from the corresponding leaf to the root. Since this path consists of at most $\lfloor \log_2 T \rfloor + 1$ nodes, the global $\ell_1$-sensitivity of the mechanism is determined as $\Delta_1 = \lfloor \log_2 T \rfloor + 1$. To ensure privacy, independent Laplace noise $Y \sim \text{Lap}(\lambda)$ is added to each node, where the scale is calibrated as:

$$\lambda = \Delta_1/\varepsilon = (\lfloor \log_2 T \rfloor + 1)/\varepsilon.$$

Second, we analyze the estimation error for a single prefix query at time $i = 1, \ldots, T$. The noisy prefix sum $\tilde{f}_i$ is computed by summing a minimal set of canonical nodes in the tree that cover the interval $[1, i]$. By the properties of binary interval tree, any prefix sum is composed of at most $\lceil \log_2 T \rceil$ such nodes. Since each node contains independent noise, the resulting estimation error $\tilde{f}_i - f_i$ is a sum of at most $k = \lceil \log_2 T \rceil$ independent Laplace random variables.

Finally, we apply the concentration bound to ensure the error estimate holds uniformly for all queries. We require the condition $|\tilde{f}_i - f_i| \le \alpha_{\text{BM}}(T, \varepsilon, \beta)$ to hold for all $i = 1, \ldots, T$ simultaneously with probability at least $1-\beta$. By the union bound, it suffices to limit the failure probability for each individual query to $\beta' = \beta/T$. According to Lemma 3.7, for a sum of $k$ Laplace variables with scale $\lambda$, setting the threshold to $\alpha_{\text{Lap}}^{(k)}(\lambda, \beta')$ guarantees that the tail probability is at most $\beta'$. Therefore, by substituting the derived parameters $\lambda$, $k$ and $\beta'$, we define the estimation error threshold as:

$$\alpha_{\text{BM}}(T, \varepsilon, \beta) = \alpha_{\text{Lap}}^{(k)}(\lambda, \beta') = \alpha_{\text{Lap}}^{(\lceil \log_2 T \rceil)}\left((\lfloor \log_2 T \rfloor + 1)/\varepsilon,\ \beta/T\right).$$

With this choice, Lemma 3.7 ensures $\Pr[|\tilde{f}_i - f_i| > \alpha_{\text{BM}}(T, \varepsilon, \beta)] \le \beta/T$ uniformly for all $i = 1, \ldots, T$, and consequently, the total failure probability is bounded by $\beta$. This completes the proof.

□

### A.3 Proof of Lemma 5.2

Lemma 5.2 (Segment Size in Private Partition). *For an $\varepsilon$-DP Private Partition over $[1, D]$ with threshold $3\ln(2D/\beta)/\varepsilon$, w.p. at least $1-\beta$, every non-final segment has a size between $\ln(2D/\beta)/\varepsilon$ and $Z = 5\ln(2D/\beta)/\varepsilon$. In particular, the expected segment size is $\Theta(Z)$.*

Proof. To prove Lemma 5.2, we analyze the mechanism of Private Partition. For brevity, let $\tau \coloneqq 3\ln(2D/\beta)/\varepsilon$ denote its comparison threshold. A segment is sealed when the noisy count reaches or exceeds the noisy threshold, i.e., when $count + Y_1 \ge \tau + Y_2$, where *count* is the current

item count, and $Y_1, Y_2 \sim \text{Lap}(1/\varepsilon)$ are independent Laplace noise variables added for a unit group size.

We adopt a global bounding strategy for the noise. Since the algorithm processes a stream of length $D$, it involves at most $2D$ independent noise draws (one for the count and one for the threshold at each step). Let $Y$ denote a generic noise variable drawn from $\text{Lap}(1/\varepsilon)$. We establish a high-probability bound on the magnitude of all such noise variables. Given a total failure probability budget $\beta$, we apply the union bound over all $2D$ potential draws. It thereby suffices to limit the failure probability of each individual draw to $\beta/(2D)$. We determine the magnitude bound $\zeta$ such that for any single noise variable $Y$, $\Pr[|Y| > \zeta] \leq \beta/(2D)$. Using the tail bound $\Pr[|Y| > z] = \exp(-\varepsilon z)$, we solve for the tightest possible $\zeta$:

$$\exp(-\varepsilon\zeta) = \beta/(2D) \implies \zeta = \ln(2D/\beta)/\varepsilon.$$

Hence, with probability at least $1 - \beta$, the magnitude of every noise variable utilized in the algorithm is bounded by $\zeta$ (i.e., $-\zeta \leq Y_i \leq \zeta$ for all $i$). Let $\mathcal{E}$ denote this high-probability event. We remark that if a more refined probabilistic analysis rather than union bound is employed here, the noise bound $\zeta$ may change; however, this does not affect our main conclusions, as they can be readily adjusted accordingly. Hence, for simplicity, all utility failure probabilities reported in this work are derived using the union bound.

Now setting the threshold to $\tau = 3\ln(2D/\beta)/\varepsilon = 3\zeta$ and conditioning on the event $\mathcal{E}$, we analyze the mechanism by treating noise as bounded error terms to first establish these properties:

***1. Minimum Segment Size.*** Suppose a segment has size $count < \zeta$. Under event $\mathcal{E}$, the noisy count is strictly upper-bounded by $2\zeta$ (since $count + Y_1 < \zeta + \zeta$), while noisy threshold is lower-bounded by $\tau - \zeta = 2\zeta$ (since $\tau + Y_2 \geq 3\zeta - \zeta$). Thus, the sealing condition is never met, as the inequality chain holds deterministically:

$$count + Y_1 < 2\zeta \leq \tau + Y_2.$$

This ensures every non-final segment has a size $\geq \zeta = \ln(2D/\beta)/\varepsilon$.

***2. Maximum Segment Size.*** Suppose the segment size reaches $count = 5\zeta$. The noisy count is lower-bounded by $4\zeta$ (since $count + Y_1 \geq 5\zeta - \zeta$), whereas the noisy threshold is upper-bounded by $4\zeta$ (since $\tau + Y_2 \leq 3\zeta + \zeta$). Thus, the sealing condition is guaranteed to be met, as the inequality chain holds deterministically:

$$count + Y_1 \geq 4\zeta \geq \tau + Y_2.$$

This ensures every non-final segment has a size $\leq Z = 5\zeta$.

***3. Expected Segment Size.*** The segment size is bounded within $[\zeta, 5\zeta]$, where $\zeta = \ln(2D/\beta)/\varepsilon$ and $Z = 5\zeta$. Since each non-final segment has size in $[Z/5, Z]$, its expectation is bounded between $\Omega(Z)$ and $O(Z)$, implying an expected segment size of $\Theta(Z)$. □

### A.4 Proof of Theorem 5.4

The utility guarantee in Theorem 5.4, namely that every non-final output segment has size in $[Z/5, Z]$, follows directly from Lemma 5.2.

We now prove the privacy guarantee. To begin, we introduce the necessary notation and conditioning event. Let $M$ denote the original Private Partition algorithm, and let $M'$ denote `DOPartition`. For clarity, we first analyze the simpler case in which two neighboring inputs induce Hamming distance 1 in the corresponding 0–1 stream, rather than the distance-2 case arising in our standard setting when the underlying sets differ in exactly one key. This simplification is only for ease of presentation. Under this temporary assumption, we study the privacy guarantee of `DOPartition` when it uses the same threshold as the original Private Partition mechanism, namely $\tau = 3\ln(2D/\delta)/\varepsilon$.

Notably, DOPartition uses a truncated version of the same Laplace noise as the original mechanism. We then translate the result back to our actual distance-2 setting, and finally invert the resulting privacy relation to justify the threshold chosen for the target $(\varepsilon, \delta)$-DP guarantee.

We define $\mathcal{E}_M$ to be the event that the magnitude of every sampled noise variable is bounded by $\ln(2D/\delta)/\varepsilon$. By Lemma 5.2, this event occurs with high probability, i.e., $\Pr[\mathcal{E}_M] \geq 1 - \delta$. Under $\mathcal{E}_M$, every non-final segment has size at least $\ln(2D/\delta)/\varepsilon$ and at most $Z = 5\ln(2D/\delta)/\varepsilon$. Based on this event, Lemma A.2 shows that the output distribution of $M'$ is exactly that of $M$ conditioned on $\mathcal{E}_M$:

Lemma A.2. *With identical threshold,* DOPartition *$(M')$ has the same output distribution as original Private Partition $(M)$ conditioned on the bounded noise event $\mathcal{E}_M$. Formally, for any output set $S$,*

$$\Pr[M'(\mathcal{X}) \in S] = \Pr[M(\mathcal{X}) \in S \mid \mathcal{E}_M].$$

Proof. We prove this by induction on the sequence of intervals defined by the keys $\mathcal{X}$. Consider the processing of a specific interval $[x_k, x_{k+1} - 1]$. Assume that strictly before processing $x_k$, both $M$ and $M'$ are in the same internal state (i.e., they share the same accumulated *count* and assign identical probabilities to any prior boundary list). Hence, both $M$ and $M'$ utilize the same Geometric distribution parameter $p$ to sample the offset for the next potential boundary. This equivalence holds because the truncated Laplace distribution used in $M'$ is mathematically defined as the Laplace distribution conditioned on the range defined by $\mathcal{E}_M$. In particular, using the same threshold $\tau$ implies that the noise bound imposed by $\mathcal{E}_M$ exactly matches the truncation range of $M'$. Let $d \in \{0, 1, \dots\}$ denote a random offset derived via inverse transform sampling as $d = \lceil \ln(u)/\ln(1-p) \rceil - 1$, where $u \sim U(0, 1)$.

In $M'$, if *count* reaches $Z = 5\ln(2D/\delta)/\varepsilon$, implying $p = 1$, a boundary $s$ is placed at $x_k$ (i.e., $\Pr[s = x_k] = 1$), followed by a dummy sample to ensure obliviousness. If $count < \ln(2D/\delta)/\varepsilon$, implying $p = 0$, the $flag$ is set to 0, indicating that no boundary is placed within current interval. Otherwise, the algorithm samples a single candidate $idx = x_k + d$. If $idx \geq x_{k+1}$, no boundary is placed in this interval, occurring with probability $\Pr[d \geq x_{k+1} - x_k]$; conversely, if $idx < x_{k+1}$, a boundary is placed at $s = idx$ with probability $\Pr[d = s - x_k]$. Upon completion of the sampling and the determination of the boundary $s$, $M'$ will advance directly to $x_{k+1}$.

Now, consider $M$ conditioned on the event $\mathcal{E}_M$. Under this condition, the bounds on the noise ensure that the threshold check fails whenever $count < \ln(2D/\delta)/\varepsilon$ and succeeds whenever $count = Z = 5\ln(2D/\delta)/\varepsilon$. Specifically, immediately upon *count* reaching $Z$ at $x_k$, the bounded noise guarantees that the noisy count meets or exceeds the noisy threshold. Thus, a boundary is placed at $x_k$ with certainty (i.e., $\Pr[s = x_k] = 1$). Conversely, when $count < \ln(2D/\delta)/\varepsilon$, the check always fails and no boundary is placed within current interval. In both cases, the behavior of $M$ matches that of $M'$.

For the sampling case, let $d$ be the distance to the first boundary sampled by $M$. First, if $d \geq x_{k+1} - x_k$, $M$ reaches $x_{k+1}$ without placing a boundary, matching the probability and behavior of $M'$. Alternatively, suppose $M$ samples a boundary at $s \in [x_k, x_{k+1} - 1]$. Upon sealing at $s$, $M$ resets its internal count to 0. In the edge case where $s = x_{k+1} - 1$, the remainder is empty, and $M$ naturally advances to $x_{k+1}$ without further sampling. Otherwise, since there are no keys in $[s+1, x_{k+1}-1]$, the count remains 0 throughout the remainder of the interval. Conditioned on $\mathcal{E}_M$, noises are bounded, so the probability of generating another boundary in the remaining interval (which would create an empty segment) is 0. Hence, $M$ places exactly one boundary at $s$ and advances to $x_{k+1}$ with probability $\Pr[d = s - x_k] \times 1$. This matches exactly with that of $M'$.

Finally, we verify the state transition to $x_{k+1}$. If a boundary is placed at $s$, $M'$ resets the count to 0 and immediately advances. Similarly, $M$ resets the count at $s$ and, under the conditioning event, deterministically generates no further boundaries within the remaining interval. Conversely, if no

boundary is placed, which occurs with identical probability for both mechanisms, they preserve the accumulated count and increment it by 1 at $x_{k+1}$. Thus, in either scenario, both mechanisms converge at $x_{k+1}$ with an identical internal state, characterized by the same resulting *count* and the equal probability assigned to the decision made at $s$.

Since both $M$ (conditioned on $\mathcal{E}_M$) and $M'$ initialize with $count = 0$ and exhibit identical step-wise transition probabilities and state updates, it follows by induction that the probability of generating any specific valid sequence of boundaries is identical. Moreover, the probability of generating any invalid partition (i.e., one containing non-final segments of size less than $\ln(2D/\delta)/\varepsilon$ or exceeding $5\ln(2D/\delta)/\varepsilon$) is identically zero for both mechanisms; this is guaranteed by design for $M'$ and enforced by the conditioning event $\mathcal{E}_M$ for $M$. Hence, the resulting probability of the output falling into any set $S$ is equal for both mechanisms, which completes the proof:

$$\Pr[M'(\mathcal{X}) \in S] = \Pr[M(\mathcal{X}) \in S \mid \mathcal{E}_M].$$

□

Equipped with the distributional equivalence in Lemma A.2, we are now ready to prove the privacy guarantee of Theorem 5.4.

Theorem 5.4. `DOPartition` *releases* $(\varepsilon, \delta)$*-DP boundaries, satisfies* $(\varepsilon, \delta)$*-DO, and produces non-final segments of size in* $[Z/5, Z]$.

Proof. We consider the original $\varepsilon$-DP Private Partition mechanism ($M$), where the threshold is calibrated to guarantee utility at least $1 - \delta$, as established in Lemma 5.2. We first analyze the privacy guarantee of `DOPartition` ($M'$) under the same threshold.

To start, let $\mathcal{X}_1$ and $\mathcal{X}_2$ be two neighboring inputs, and let $S$ be an arbitrary subset of the output space of $M'$. By Lemma A.2, the output distribution of $M'$ on $S$ is identical to that of $M$ conditioned on the event $\mathcal{E}_M$. For the input $\mathcal{X}_1$, we derive:

$$\begin{aligned}\Pr[M'(\mathcal{X}_1) \in S] &= \Pr[M(\mathcal{X}_1) \in S \mid \mathcal{E}_M] \\ &\leq \frac{\Pr[(M(\mathcal{X}_1) \in S) \cap \mathcal{E}_M]}{\Pr[\mathcal{E}_M]} \\ &\leq \frac{\Pr[M(\mathcal{X}_1) \in S]}{1 - \delta}.\end{aligned}$$

The last inequality holds for two reasons. First, regarding the numerator: the event $(M(\mathcal{X}_1) \in S) \cap \mathcal{E}_M$ is a subset of $M(\mathcal{X}_1) \in S$. So, the probability of the intersection is bounded by $\Pr[M(\mathcal{X}_1) \in S]$. Second, for the denominator: the noise bound event $\mathcal{E}_M$ occurs with probability $\Pr[\mathcal{E}_M] \geq 1 - \delta$, as established in Lemma 5.2

Next, we leverage the $\varepsilon$-DP guarantee of the original mechanism $M$. For neighboring datasets $\mathcal{X}_1$ and $\mathcal{X}_2$, it holds that:

$$\Pr[M(\mathcal{X}_1) \in S] \leq e^{\varepsilon} \cdot \Pr[M(\mathcal{X}_2) \in S].$$

Substituting this upper bound into the previous inequality yields:

$$\Pr[M'(\mathcal{X}_1) \in S] \leq \frac{e^{\varepsilon}}{1 - \delta} \cdot \Pr[M(\mathcal{X}_2) \in S]. \tag{1}$$

To relate the term $\Pr[M(\mathcal{X}_2) \in S]$ back to $M'$, we decompose the probability based on the occurrence of the event $\mathcal{E}_M$:

$$\begin{aligned}\Pr[M(\mathcal{X}_2) \in S] &= \Pr[M(\mathcal{X}_2) \in S \cap \mathcal{E}_M] \\ &\quad + \Pr[M(\mathcal{X}_2) \in S \cap \neg\, \mathcal{E}_M] \\ &= \Pr[M(\mathcal{X}_2) \in S \mid \mathcal{E}_M] \cdot \Pr[\mathcal{E}_M] \\ &\quad + \Pr[M(\mathcal{X}_2) \in S \cap \neg\, \mathcal{E}_M].\end{aligned}$$

Applying Lemma A.2, we replace the conditional probability in the first term with $\Pr[M'(\mathcal{X}_2) \in S]$. Since the probability of the conditioning event $\mathcal{E}_M$ is at most 1, the first term is simply upper-bounded by $\Pr[M'(\mathcal{X}_2) \in S]$. For the second term, we observe that the joint probability is trivially bounded by the event probability itself: $\Pr[M(\mathcal{X}_2) \in S \cap \neg\, \mathcal{E}_M] \le \Pr[\neg\, \mathcal{E}_M]$. Here, $\neg\, \mathcal{E}_M$ denotes the failure event in which at least one noise sample in $M$ exceeds the bound $\ln(2D/\delta)/\varepsilon$, with probability $\Pr[\neg\, \mathcal{E}_M] \le \delta$. Thus, we obtain:

$$\begin{aligned}\Pr[M(\mathcal{X}_2) \in S] &\le \Pr[M'(\mathcal{X}_2) \in S] \cdot 1 + \Pr[\neg\, \mathcal{E}_M] \\ &\le \Pr[M'(\mathcal{X}_2) \in S] + \delta.\end{aligned}$$

Finally, we substitute this upper bound back into Equation (1):

$$\begin{aligned}\Pr[M'(\mathcal{X}_1) \in S] &\le \frac{e^\varepsilon}{1-\delta}\left(\Pr[M'(\mathcal{X}_2) \in S] + \delta\right) \\ &= \frac{e^\varepsilon}{1-\delta}\Pr[M'(\mathcal{X}_2) \in S] + \frac{\delta e^\varepsilon}{1-\delta} \\ &= \left(e^\varepsilon + \frac{\delta e^\varepsilon}{1-\delta}\right)\Pr[M'(\mathcal{X}_2) \in S] + \frac{\delta e^\varepsilon}{1-\delta} \\ &\le e^\varepsilon \Pr[M'(\mathcal{X}_2) \in S] + \frac{2\delta e^\varepsilon}{1-\delta}.\end{aligned}$$

The last inequality holds since $\Pr[M'(\mathcal{X}_2) \in S] \le 1$.

Hence, assuming `DOPartition` employs the same threshold as original $\varepsilon$-DP Private Partition, it satisfies $(\varepsilon, \frac{2\delta e^\varepsilon}{1-\delta})$-DP. Note that its segment identification differs only in taking exactly one sample per interval and using same-scale but truncated Laplace noise.

We now return to the neighboring relation in our actual setting. A single write-operation difference between two neighboring inputs induces Hamming distance 2 in the corresponding 0–1 stream. Therefore, by group privacy, it suffices to instantiate the original Private Partition mechanism under distance 1 with privacy budget $\varepsilon/2$. Concretely, this changes its Laplace noise from $\mathrm{Lap}(1/\varepsilon)$ to $\mathrm{Lap}(2/\varepsilon)$, and correspondingly changes the threshold from $\tau = 3\zeta = 3\ln(2D/\delta)/\varepsilon$ to $\tau = 3\zeta = 6\ln(2D/\delta)/\varepsilon$. Under this instantiation, the original Private Partition mechanism guarantees $\varepsilon$-DP together with utility at least $1-\delta$. We then instantiate `DOPartition` with the same threshold and the same noise scale, except that it uses the truncated version of this Laplace noise. The above argument thus implies that `DOPartition` satisfies $(\varepsilon, \frac{2\delta e^\varepsilon}{1-\delta})$-DP in our setting.

Finally, if we want `DOPartition` to satisfy exact standard $(\varepsilon, \delta)$-DP, we can calibrate the utility failure probability of the underlying Private Partition mechanism more carefully. Let $\delta'$ denote the failure probability used there. Since the above analysis yields an additive privacy loss of $2\delta' e^\varepsilon/(1-\delta')$, we set $2\delta' e^\varepsilon/(1-\delta') = \delta$, which gives $\delta' = \delta/(2e^\varepsilon + \delta)$. Substituting this value into the threshold formula yields $\tau = 3\zeta = 6\ln(2D(2e^\varepsilon + \delta)/\delta)/\varepsilon$, which is exactly the threshold adopted in our design. Therefore, under this choice of threshold, `DOPartition` releases $(\varepsilon, \delta)$-DP boundaries.

Moreover, since DOPartition performs exactly one sampling per interval and FOCompact is FO, its execution depends only on public parameters and the released boundaries. By post-processing, its execution is thus $(\varepsilon, \delta)$-DP, establishing $(\varepsilon, \delta)$-DO for DOPartition.

□

Received 17 April 2026; revised 19 August 2026; accepted 12 September 2026